

# 數位邏輯—使用 Verilog HDL 設計

邏輯電路簡介

實作技術

邏輯函數之最佳化實作

數值表示法與數學電路

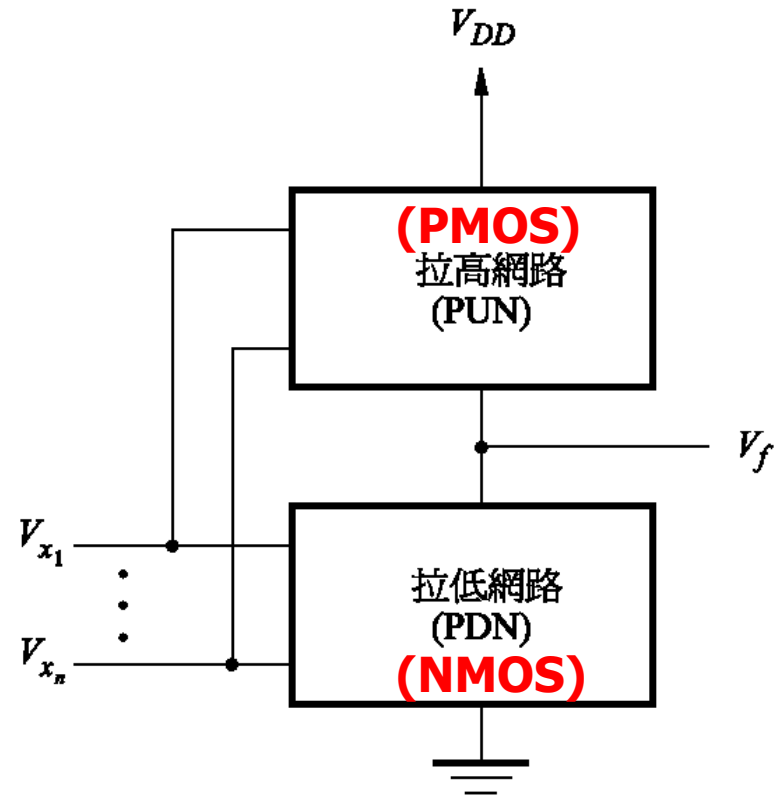


**McGraw-Hill Education**

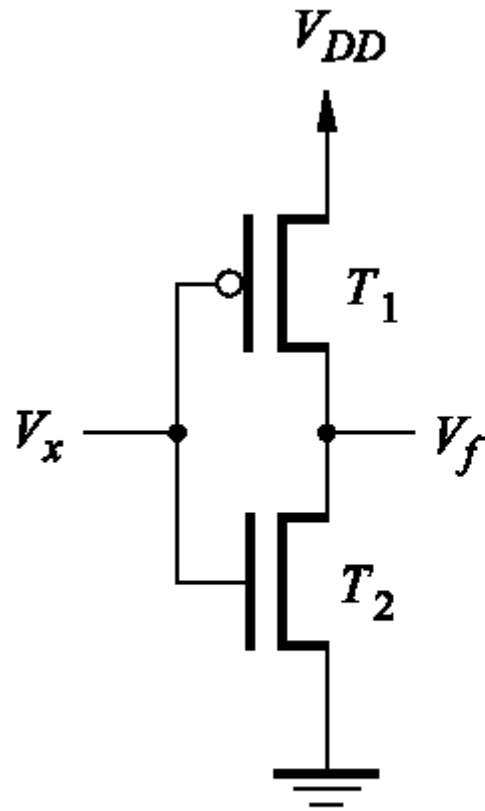
# CMOS邏輯閘

## CMOS Logic Gates

- 在 NMOS 電路中，藉由 NMOS 電晶體和作為電阻的拉高元件，可以實現邏輯電路。我們將涉及 NMOS 電晶體的電路稱為**拉低網路** (pull-down network, PDN)。
- **CMOS 電路的觀念是將拉高元件以 PMOS 電晶體建構的拉高網路 (pull-up network, PUN) 取代，使得函數由 PDN 和 PUN 網路所組成，互為互補。**
- **PDN 與 PUN 有同樣數目的電晶體，因此互為對偶 (dual)。**



圖：CMOS電路的架構。



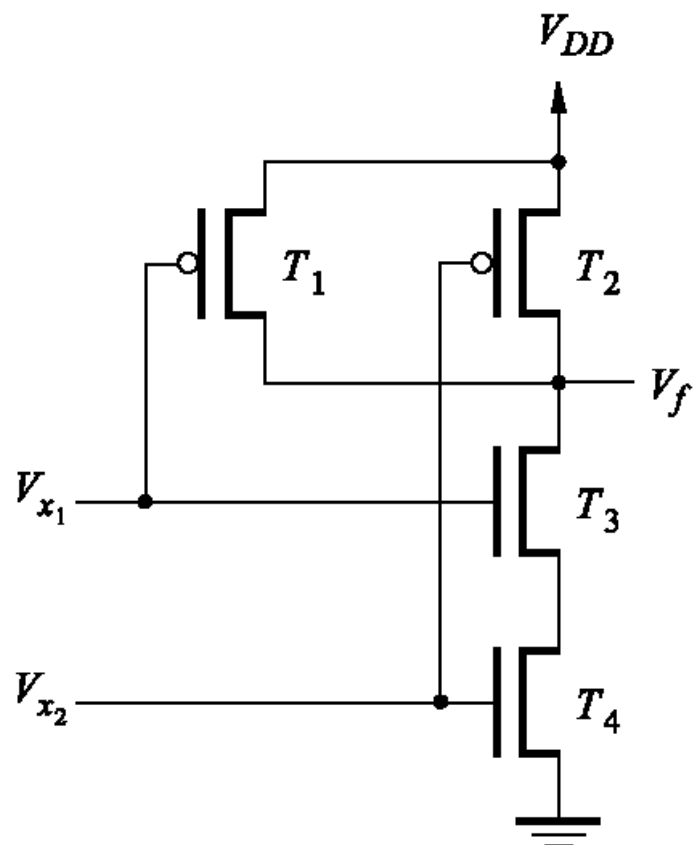
(a) 電路

$$f = \bar{x}$$

| $x$ | $T_1$ | $T_2$ | $f$ |
|-----|-------|-------|-----|
| 0   | on    | off   | 1   |
| 1   | off   | on    | 0   |

(b) 真值表和電晶體狀態

圖：NOT邏輯閘的CMOS實現。



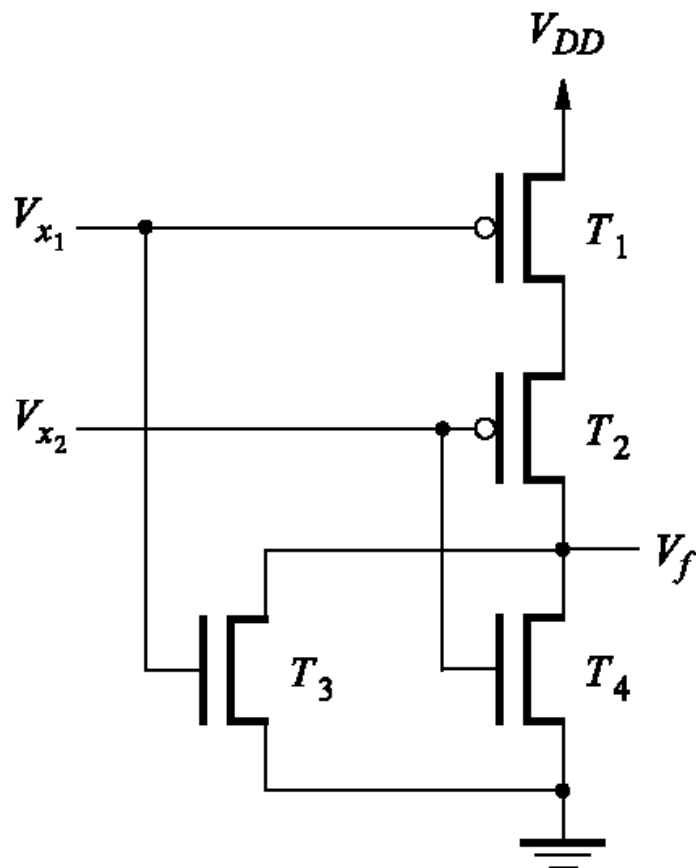
(a) 電路

$$f = \overline{x_1 x_2} = \overline{x_1} + \overline{x_2}$$

| $x_1$ | $x_2$ | $T_1$ | $T_2$ | $T_3$ | $T_4$ | $f$ |
|-------|-------|-------|-------|-------|-------|-----|
| 0     | 0     | on    | on    | off   | off   | 1   |
| 0     | 1     | on    | off   | off   | on    | 1   |
| 1     | 0     | off   | on    | on    | off   | 1   |
| 1     | 1     | off   | off   | on    | on    | 0   |

(b) 真值表和電晶體狀態

圖：NAND邏輯閘的CMOS實現。



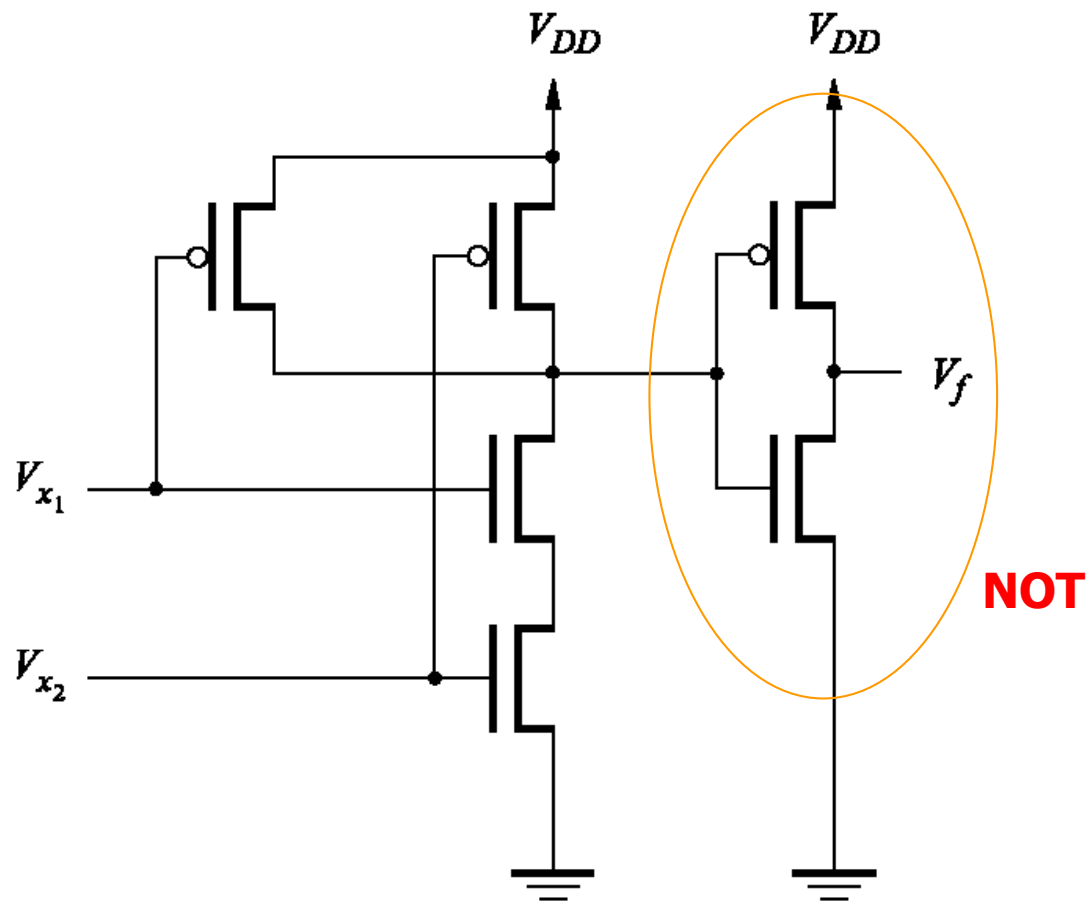
(a) 真值表

$$f = \overline{x_1 + x_2} = \overline{x_1} \overline{x_2}$$

| $x_1$ | $x_2$ | $T_1$ | $T_2$ | $T_3$ | $T_4$ | $f$ |
|-------|-------|-------|-------|-------|-------|-----|
| 0     | 0     | on    | on    | off   | off   | 1   |
| 0     | 1     | on    | off   | off   | on    | 0   |
| 1     | 0     | off   | on    | on    | off   | 0   |
| 1     | 1     | off   | off   | on    | on    | 0   |

(b) 真值表和電晶體狀態

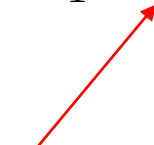
圖：NOR邏輯閘的CMOS實現。



圖：AND邏輯閘的CMOS實現。

# 範例

## ■ 考慮下列函數

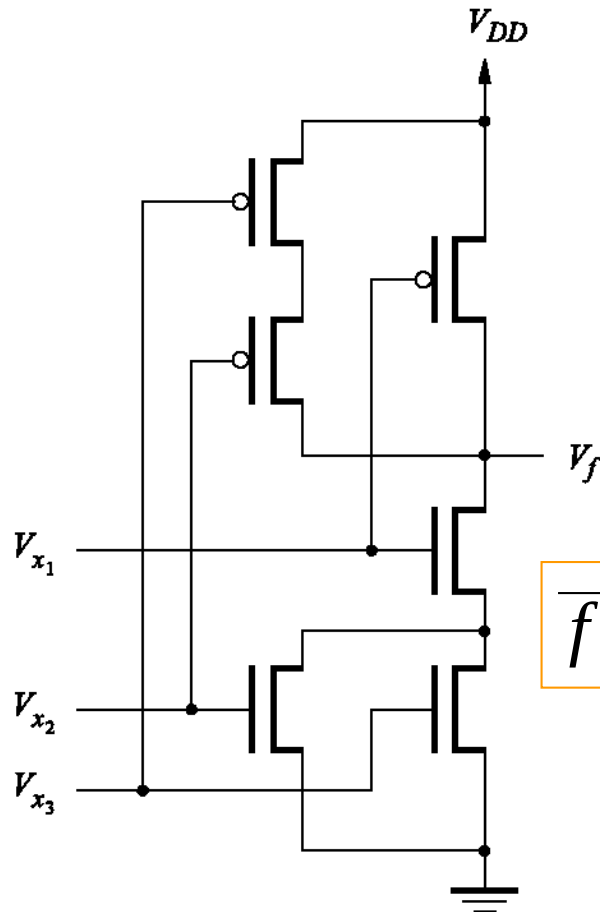
$$f = \bar{x}_1 + \bar{x}_2 \bar{x}_3$$


- 因為所有變數都以互補形式出現，我們可以直接得到 PUN。先將  $x_2$  和  $x_3$  控制的 PMOS 電晶體串聯，再與  $x_1$  所控制的 PMOS 電晶體並聯。

$$\overline{f} = \overline{\bar{x}_1 + \bar{x}_2 \bar{x}_3} = x_1(x_2 + x_3)$$

- 此式給定的 PDN 為  $x_2$  和  $x_3$  控制的 NMOS 電晶體先並聯，再與  $x_1$  控制的 NMOS 電晶體串聯。

# 範例



$$f = \bar{x}_1 + \bar{x}_2 \bar{x}_3$$

$$\bar{f} = \overline{\bar{x}_1 + \bar{x}_2 \bar{x}_3} = x_1(x_2 + x_3)$$

圖：範例電路。



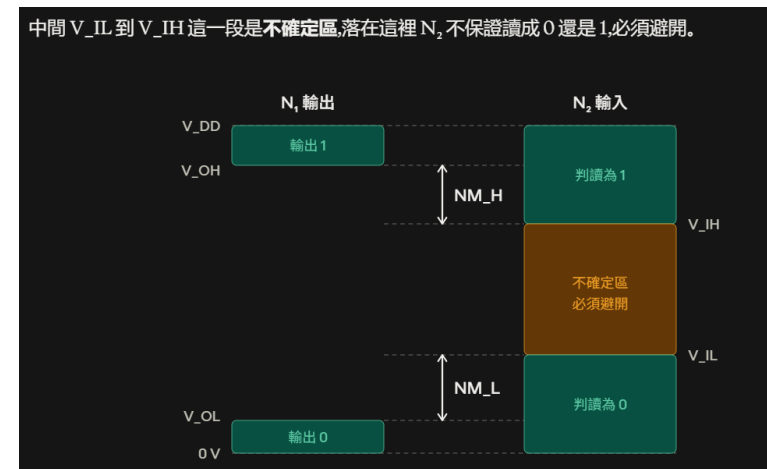
# 雜訊容限 Noise Margin

- 考慮圖(a)的兩個NOT邏輯閘。左邊與右邊的邏輯閘分別為 $N_1$ 與 $N_2$ 。電子電路常常會遇到隨機的干擾，稱為**雜訊** (noise)，可能會改變邏輯閘 $N_1$ 的輸出電壓值。
- 可以容許雜訊而不影響電路正確運作的能力稱為**雜訊容限** (noise margin)。我們定義低雜訊容限為

$$NM_L = V_{IL} - V_{OL}$$

- 高雜訊容限定義為

$$NM_H = V_{OH} - V_{IH}$$



曲線分三段看

左邊平坦段 ( $V_x < V_T$ )

$V_x$  還沒超過 NMOS 的臨界電壓  $V_T$ , 下面那顆 NMOS 完全不導通, 只有 PMOS 在拉 → 輸出被牢牢拉到  $V_{DD}$ , 所以  $V_{OH} = V_{DD}$ .

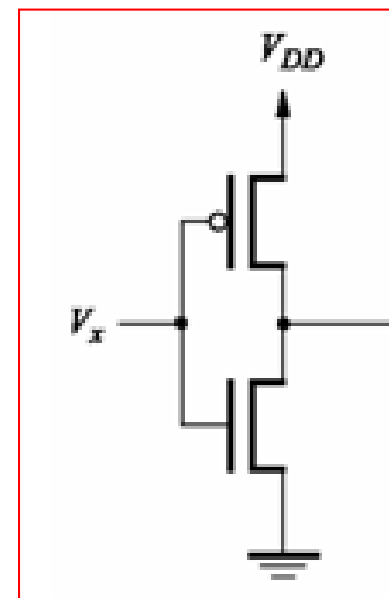
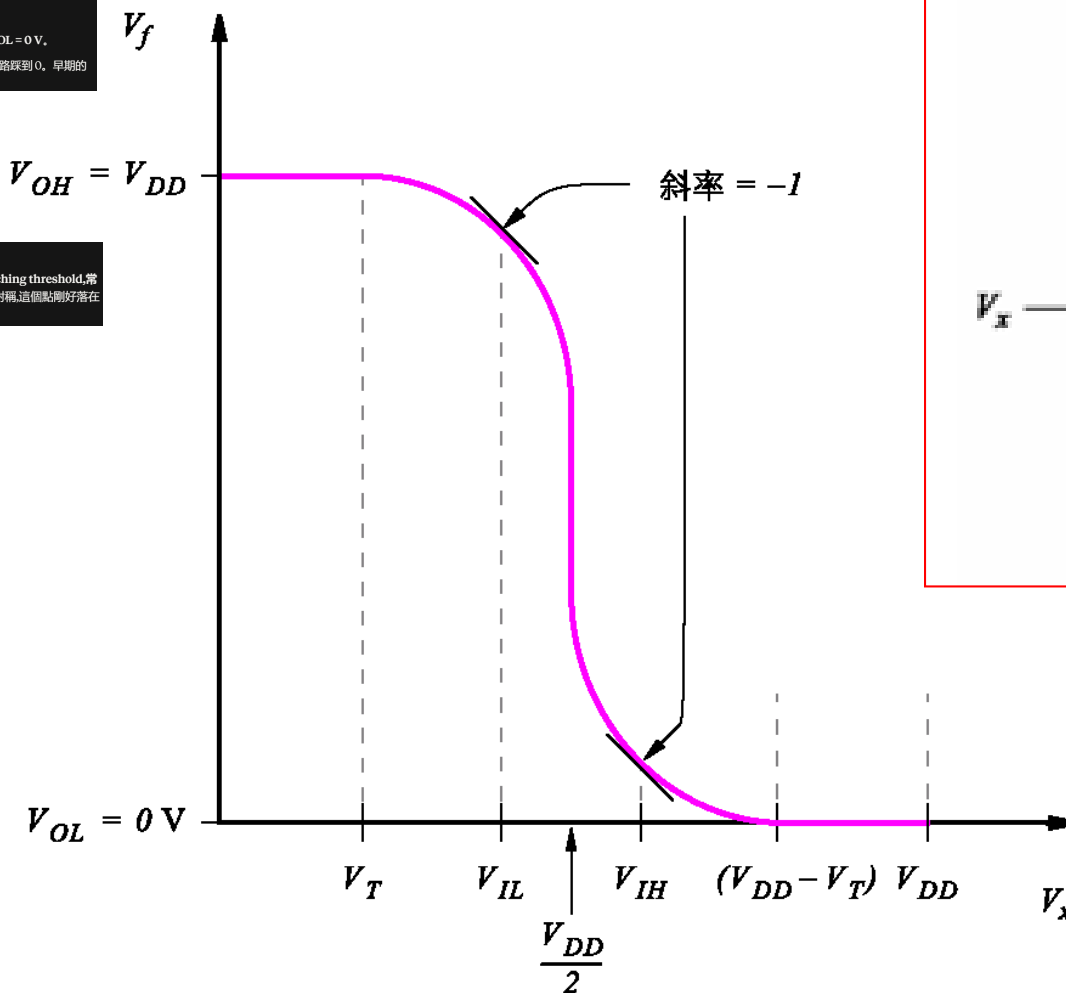
右邊平坦段 ( $V_x > V_{DD} - V_T$ )

PMOS 的開源電壓不足, PMOS 截止, 只有 NMOS 在拉 → 輸出被拉到地, 所以  $V_{OL} = 0V$ .

這兩段是 CMOS 的最大優勢, 輸出能做到滿擺 (rail-to-rail), 一路頂到  $V_{DD}$ , 一路踩到 0。早期的 NMOS 邏輯或 TTL 做不到, 雜訊容限就差很多。

中間陡降段

兩顆電晶體同時導通, 輸出快速從高翻到低。箭頭指的  $V_{DD}/2$  是切換點 (switching threshold, 常記作  $V_M$ ), 也就是曲線最陡的位置。如果 PMOS 和 NMOS 的驅動能力設計成對稱, 這個點剛好落在  $V_{DD}/2$ 。

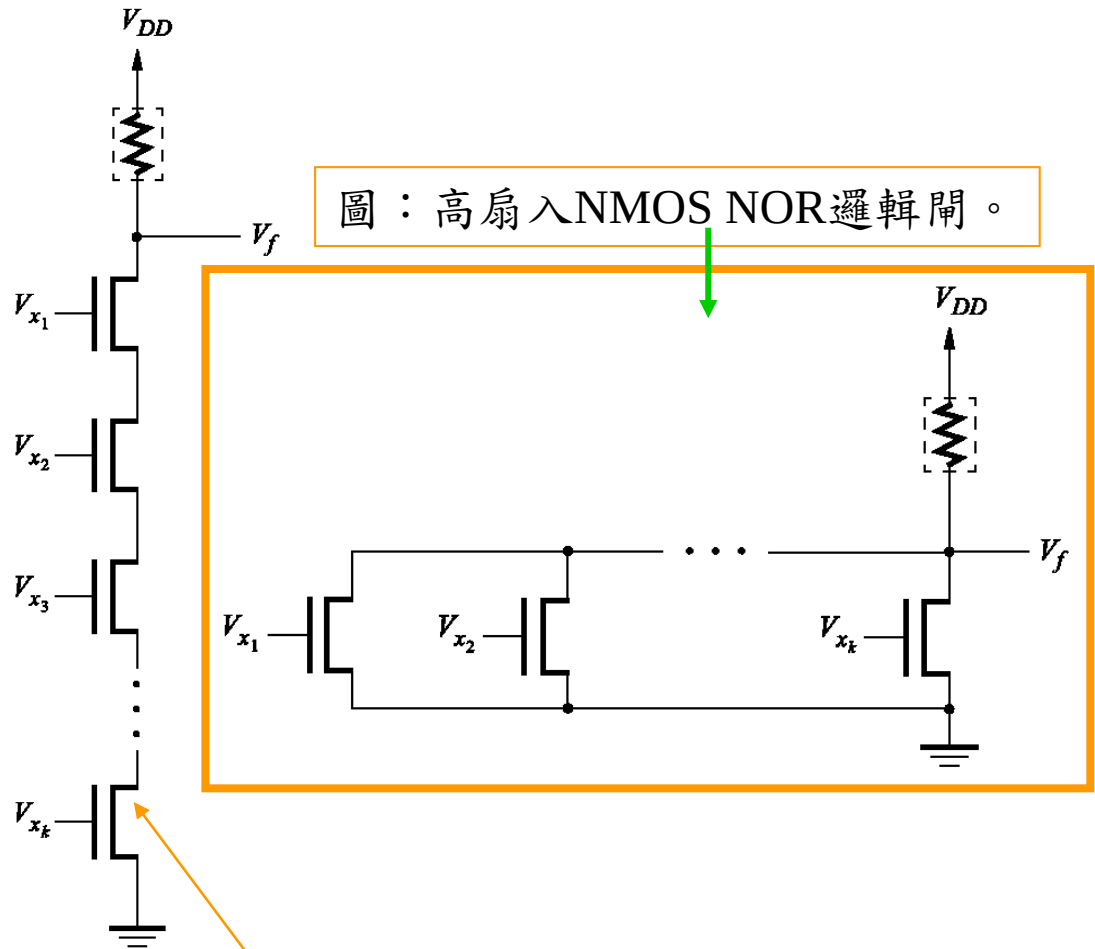


圖：CMOS反向器的電壓轉換特徵。

# 邏輯閘的扇入和扇出

## Fan-in and Fan-out in Logic Gates

- 邏輯閘的扇入 (fan-in)  
定義為邏輯閘的輸入個數。
- 增加輸入個數超過某個值之後就變得不切實際，這與邏輯閘如何建構有關。例如，考慮右圖的 NMOS NAND 邏輯閘，有  $k$  個輸入。
- 此時  $C$  為位於邏輯閘輸出的等效電容，包含  $k$  個電晶體所貢獻的寄生電容。

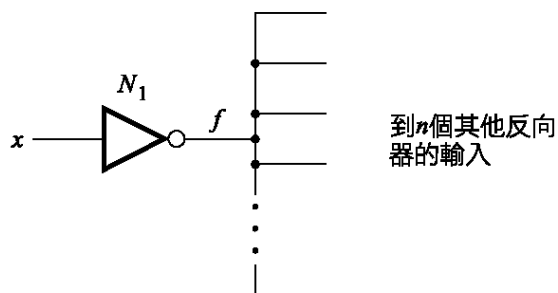


圖：高扇入NMOS NOR邏輯閘。

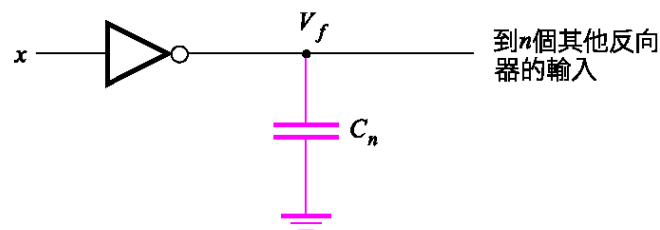
圖：高扇入NMOS NAND邏輯閘。

# 扇出 Fan-out

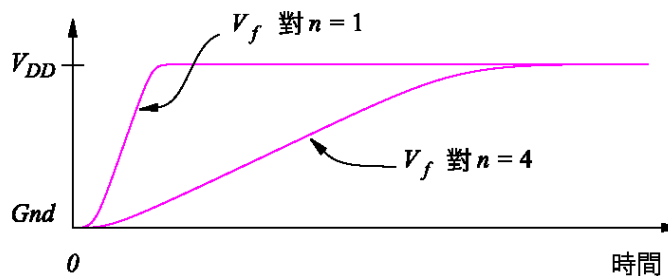
- 邏輯閘驅動其他邏輯閘的個數稱為扇出 (fan-out)。
- 圖(a)為扇出的例子，有個反向器 $N_1$ 驅動 $n$ 個其他反向器的輸入。
- 在圖中 (b) 部分， $n$ 個反向器以一個大電容 $C_n$ 表示。
- 圖© 說明 $n$ 如何影響傳遞延遲。



(a) 驅動 $n$ 個其他反向器的反向器



(b) 時序用途的等效電路



(c) 不同 $n$ 值的傳遞時間

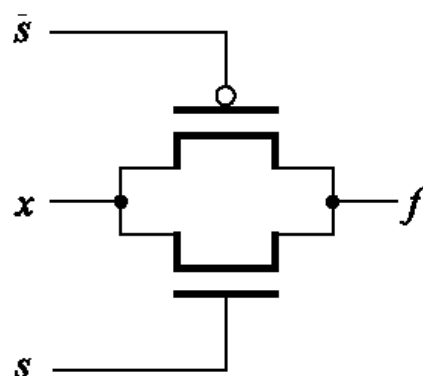
圖：傳遞延遲的扇出的影響。

# 傳輸閘

## Transmission Gates

- 將 NMOS 和 PMOS 電晶體組合在單一開關，使其能夠驅動輸出端至低值或高值都很好。右圖(a)為此電路，稱為 **傳 輸 閘** (transmission gate)。
- 如右圖中 (b) 部分與 (c) 部分所示，就像開關一般從  $x$  連接到  $f$ 。
- 傳輸閘的圖形符號如右圖(d)所示。

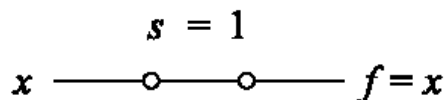
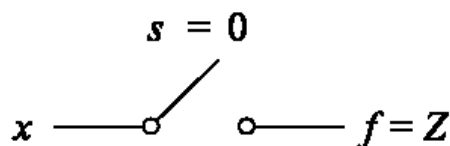
圖：傳輸閘。



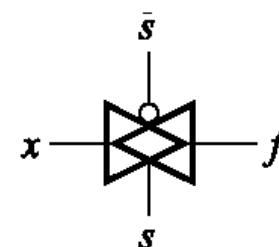
(a) 電路

| $s$ | $f$ |
|-----|-----|
| 0   | Z   |
| 1   | $x$ |

(b) 真值表



(c) 等效電路



(d) 圖形符號

# Appendix: Complementary switch

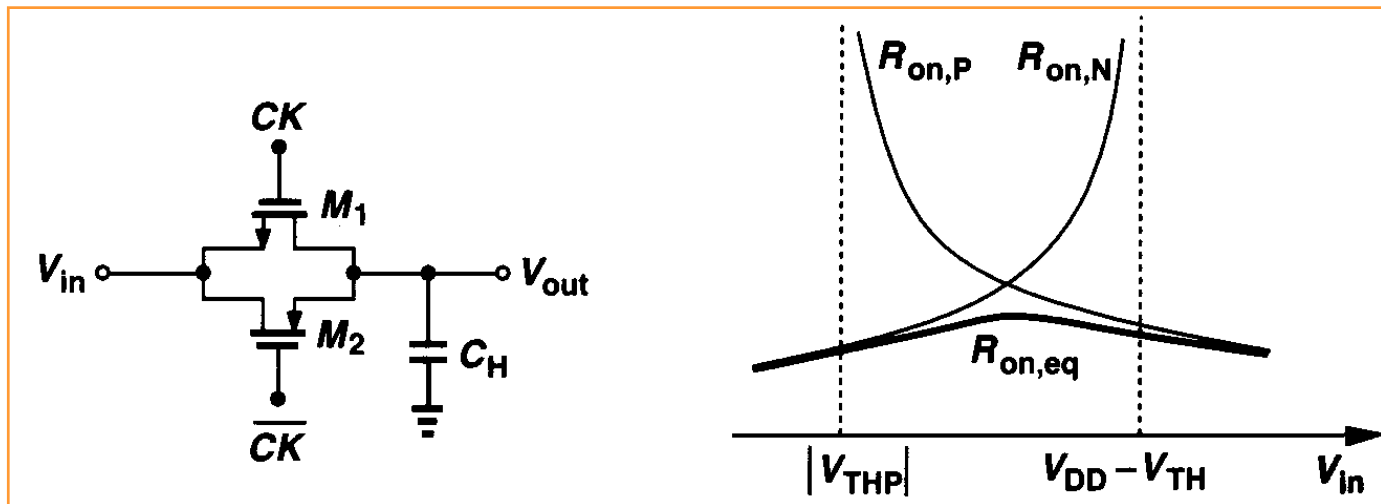
- An equivalent resistance of complementary is

$$R_{on,eq} = R_{on,N} \parallel R_{on,P} = \frac{1}{\mu_n C_{ox} (W/L)_N (V_{DD} - V_{in} - V_{THN})} \parallel \frac{1}{\mu_p C_{ox} (W/L)_P (V_{in} - |V_{THP}|)}$$

$$= \frac{1}{\mu_n C_{ox} \left(\frac{W}{L}\right)_N (V_{DD} - V_{THN}) - \left[ \mu_n C_{ox} \left(\frac{W}{L}\right)_N - \mu_p C_{ox} \left(\frac{W}{L}\right)_P \right] V_{in} - \mu_p C_{ox} \left(\frac{W}{L}\right)_P |V_{THP}|}$$

**Important !!!**

- If  $\mu_n C_{ox} (W/L)_N = \mu_p C_{ox} (W/L)_P$ , then  **$R_{on,eq}$  is independent of the input level.** That is, *the complementary switch is much less variation than that corresponding to each switch alone.*



# XOR邏輯閘(邏輯互斥或閘)

## Exclusive-OR Gates

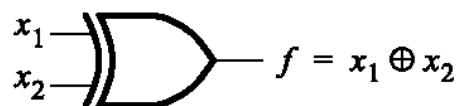
- 此元件實現XOR函數，定義如次頁圖(a)所示。此函數的真值表和OR函數很類似，差別只是兩個輸入皆為1時 $f = 0$ 。由於此相似性，此函數稱為**互斥OR**，通常簡寫為**XOR**。實作XOR的邏輯閘的圖形符號如圖(b)部分所示。
- XOR運算通常以 $\oplus$ 符號表示。可以用乘積總和形式表示

$$x_1 \oplus x_2 = \bar{x}_1 x_2 + x_1 \bar{x}_2$$

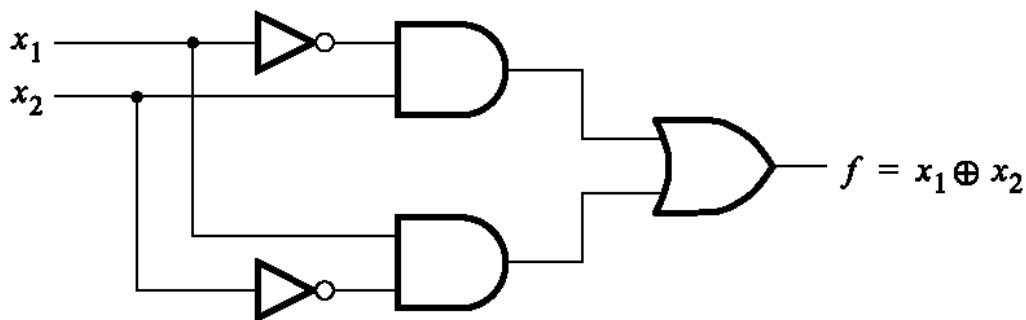
- 圖(c)的邏輯實作電路。我們可以知道每個AND和OR邏輯閘都需要六個電晶體，而NOT邏輯閘需要兩個電晶體。因此，以CMOS技術需要22個電晶體來實作此電路。藉由使用傳輸閘，我們可以大大減少所需的電晶體個數。
- 圖(d)為XOR邏輯閘的傳輸閘電路，用了兩個傳輸閘和兩個反向器。當 $x_1 = 0$ 時，頂端傳輸閘設定輸出 $f$ 為 $x_2$ 。
- 當 $x_1 = 1$ 時，底部傳輸閘將 $f$ 設成 $\bar{x}_2$ 。

| $x_1$ | $x_2$ | $f = x_1 \oplus x_2$ |
|-------|-------|----------------------|
| 0     | 0     | 0                    |
| 0     | 1     | 1                    |
| 1     | 0     | 1                    |
| 1     | 1     | 0                    |

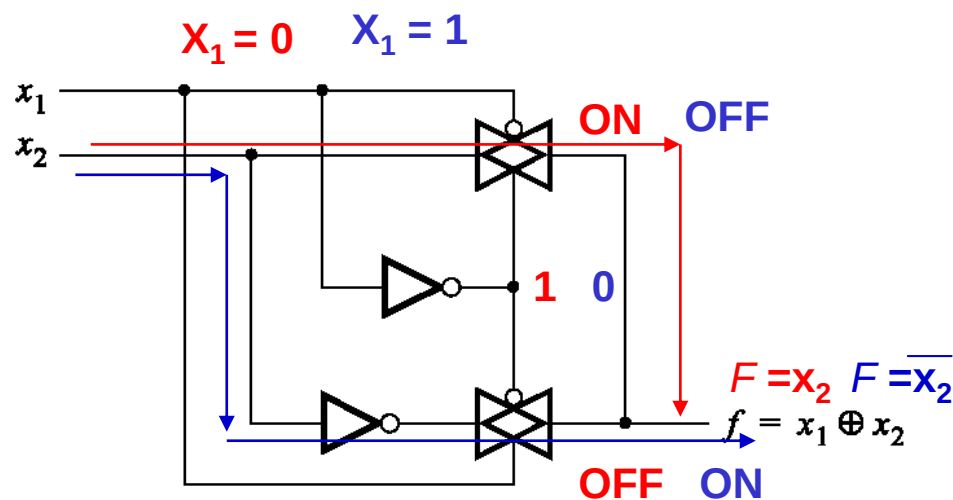
(a) 真值表



(b) 圖形符號



(c) 乘積總和實作



(d) CMOS 實作

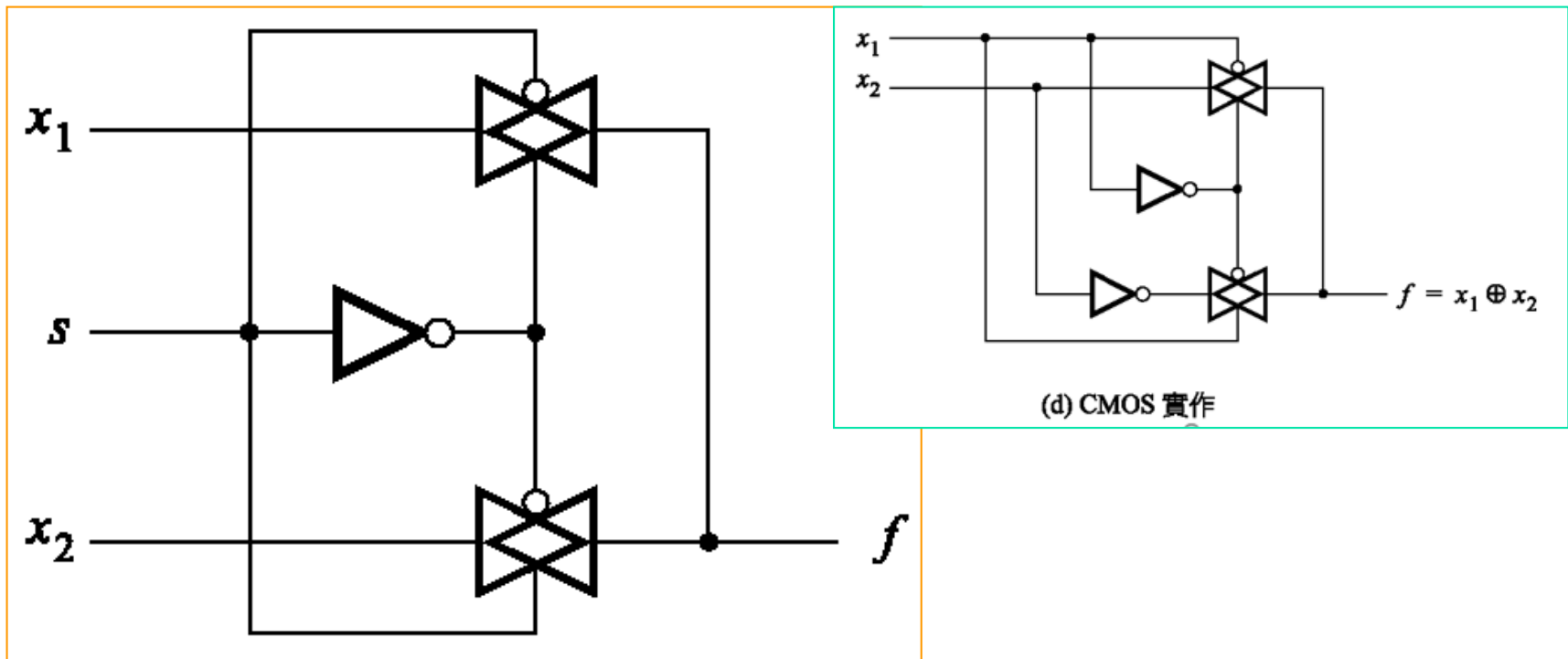
圖：XOR邏輯閘。



# 多工器電路

## Multiplexer Circuit

- 類似的架構：以傳輸閘實現二對一多工器，如下圖所示。



圖：以傳輸閘建構的二對一多工器。

# 布林代數：二變數與三變數特性

## Two- and Three-Variables Properties

$$10a \quad x \cdot y = y \cdot x$$

交換性 (Commutative)

$$10b \quad x + y = y + x$$

$$11a \quad x \cdot (y \cdot z) = (x \cdot y) \cdot z$$

聯結性 (Associative)

$$11b \quad x + (y + z) = (x + y) + z$$

$$12a \quad x \cdot (y + z) = x \cdot y + x \cdot z$$

分配性 (Distributive)

$$12b \quad x + y \cdot z = (x + y) \cdot (x + z) \quad \text{--- Appendix}$$

$$13a \quad x + x \cdot y = x$$

吸收性 (Absorption)

$$13b \quad x \cdot (x + y) = x$$

$$13(b): x \bullet (x + y) = x \bullet x + x \bullet y = x + x \bullet y = x$$

$$14a \quad x \cdot y + x \cdot \bar{y} = x$$

組合性 (Combining)

$$14b \quad (x + y) \cdot (x + \bar{y}) = x$$

$$14(a): x \bullet y + x \bullet \bar{y} = x \bullet (y + \bar{y}) = x \bullet 1 = x$$

$$15a \quad \overline{x \cdot y} = \bar{x} + \bar{y}$$

狄摩根定理 (DeMorgan's theorem)

$$15b \quad \overline{x + y} = \bar{x} \cdot \bar{y}$$

Appendix

$$16a \quad x + \bar{x} \cdot y = x + y \quad \text{--- Appendix}$$

Appendix

$$16b \quad x \cdot (\bar{x} + y) = x \cdot y$$

$$16(b): x \bullet (\bar{x} + y) = x \bullet \bar{x} + x \bullet y = x \bullet y$$

## Appendix: 二變數與三變數特性

$$12(b): (x+y) \bullet (x+z) = x \bullet (x+z) + y \bullet (x+z) = x \bullet x + x \bullet z + y \bullet x + y \bullet z \\ = (x + x \bullet z + y \bullet x) + y \bullet z = x + y \bullet z$$

$$14(b): (x+y) \bullet (x+\bar{y}) = x \bullet x + x \bullet \bar{y} + y \bullet x + y \bullet \bar{y} = x + x \bullet (y+\bar{y}) + 0 \\ = x + x \bullet 1 + 0 = x$$

$$16(a): x + \bar{x} \bullet y = x + y \\ \Rightarrow \overline{x + \bar{x} \bullet y} = \bar{x} \bullet \overline{(\bar{x} \bullet y)} = \bar{x} \bullet (\overline{\bar{x}} + \bar{y}) = \bar{x} \bullet \bar{\bar{x}} + \bar{x} \bullet \bar{y} \\ = 0 + \bar{x} \bullet \bar{y} = \overline{x + y} \\ \Rightarrow \overline{\overline{x + \bar{x} \bullet y}} = \overline{\overline{x + y}} \Rightarrow x + \bar{x} \bullet y = x + y$$

# 邏輯證明的範例

- 驗證下列邏輯方程式的正確性

$$(x_1 + x_3) \cdot (\bar{x}_1 + \bar{x}_3) = x_1 \cdot \bar{x}_3 + \bar{x}_1 \cdot x_3$$

- 等號左邊可以進行下列運算，使用特性12a 的分配性

$$\text{LHS} = (x_1 + x_3) \cdot \bar{x}_1 + (x_1 + x_3) \cdot \bar{x}_3$$

- 再次運用分配性，得到

$$\text{LHS} = x_1 \cdot \bar{x}_1 + x_3 \cdot \bar{x}_1 + x_1 \cdot \bar{x}_3 + x_3 \cdot \bar{x}_3$$

- 注意分配性使我們可以對括號中的各項做AND運算，如同一般的代數。接下來，根據定理8a，與都等於0。因此，

$$\text{LHS} = 0 + x_3 \cdot \bar{x}_1 + x_1 \cdot \bar{x}_3 + 0$$

- 根據6a可以得到

$$\text{LHS} = x_3 \cdot \bar{x}_1 + x_1 \cdot \bar{x}_3$$

- 最後，使用交換性，10a和10b，變成

$$\text{LHS} = x_1 \cdot \bar{x}_3 + \bar{x}_1 \cdot x_3$$

- 與等式右邊相同。

# 設計實例：多工器電路

## Multiplexer Circuit

- 這個電路將有三個輸入： $x_1$ 、 $x_2$ 和 $s$ 。
- 若 $s = 0$ ，則電路輸出值與輸入 $x_1$ 相同；若 $s = 1$ ，則電路輸出值與輸入 $x_2$ 相同。
- 根據真值表，得到乘積總和表示式為

$$f(s, x_1, x_2) = \bar{s}x_1\bar{x}_2 + \bar{s}x_1x_2 + s\bar{x}_1x_2 + sx_1x_2$$

- 使用分配性，上式可以寫成

$$f = \bar{s}x_1(\bar{x}_2 + x_2) + s(\bar{x}_1 + x_1)x_2$$

- 套用定理8b，產生

$$f = \bar{s}x_1 \cdot 1 + s \cdot 1 \cdot x_2$$

- 最後，定理6a 得到

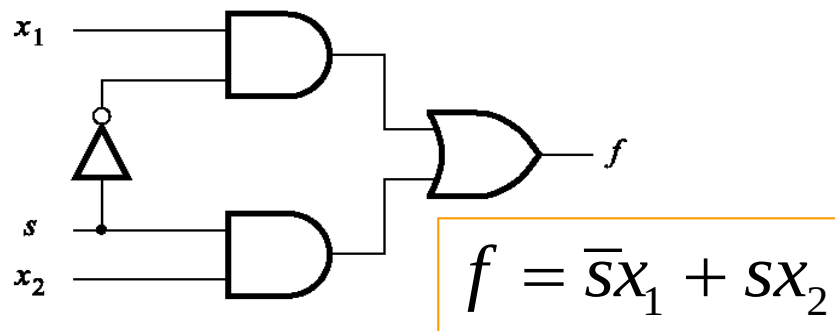
$$f = \bar{s}x_1 + sx_2$$

| $s \ x_1 \ x_2$ | $f(s, x_1, x_2)$ |
|-----------------|------------------|
| 0 0 0           | 0                |
| 0 0 1           | 0                |
| 0 1 0           | 1                |
| 0 1 1           | 1                |
| 1 0 0           | 0                |
| 1 0 1           | 1                |
| 1 1 0           | 0                |
| 1 1 1           | 1                |

(a) 真值表

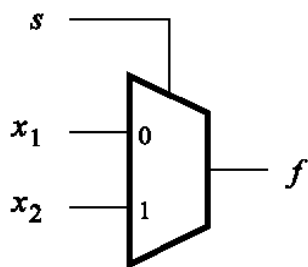
| $s$ | $x_1$ | $x_2$ | $f(s, x_1, x_2)$ |
|-----|-------|-------|------------------|
| 0   | 0     | 0     | 0                |
| 0   | 0     | 1     | 0                |
| 0   | 1     | 0     | 1                |
| 0   | 1     | 1     | 1                |
| 1   | 0     | 0     | 0                |
| 1   | 0     | 1     | 1                |
| 1   | 1     | 0     | 0                |
| 1   | 1     | 1     | 1                |

(a) 真值表



(b) 電路

若  $s=0$  則  $f=x_1$ ；若  $s=1$  則  $f=x_2$ 。



(c) 圖形符號

| $s$ | $f(s, x_1, x_2)$ |
|-----|------------------|
| 0   | $x_1$            |
| 1   | $x_2$            |

(d) 較精簡的真值表表示

圖：多工器的實作。

# 硬體描述語言

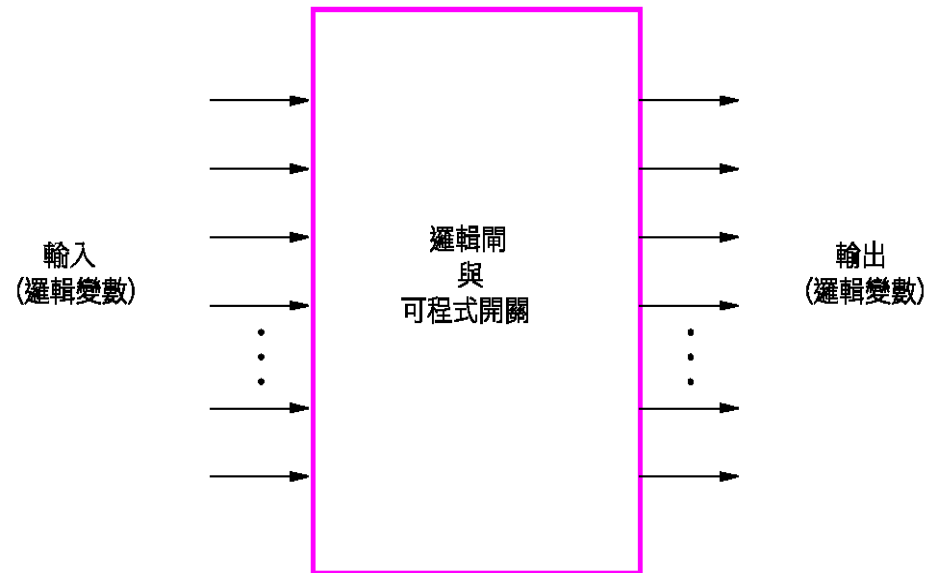
## Hardware Description Language

- **硬體描述語言** (hardware description language, HDL) 很類似於傳統電腦程式語言，除了HDL是用來描述硬體，而程式是在電腦上執行。
- 由電子電機工程師協會 (Institute of Electrical and Electronics Engineers, IEEE)所認可的語言。
- 有兩種HDL是IEEE標準：**Verilog HDL**和**VHDL (Very High Speed Integrated Circuit Hardware Description Language)**。

# 可程式邏輯元件

## Programmable Logic Devices

- 每個7400系列元件所提供的功能是固定的，無法修改以符合特殊設計狀況。除此之外，受限於每個晶片只包含少數的邏輯閘，用這些晶片來建構大型電路是相當沒有效率的。
- 這種含有較多邏輯電路且架構非固定的晶片在1970年代首度出現，稱為可程式邏輯元件 (programmable logic device, PLD)。
- PLD是用來實作邏輯電路的一般性晶片。它包含許多邏輯電路元件，可以不同的方式實作。
- PLD可以視為包含邏輯閘和可程式開關的『黑盒子 (black box)』，如右圖所示。可程式開關可將PLD內部的邏輯閘連接起來，實作出任何所需的電路。



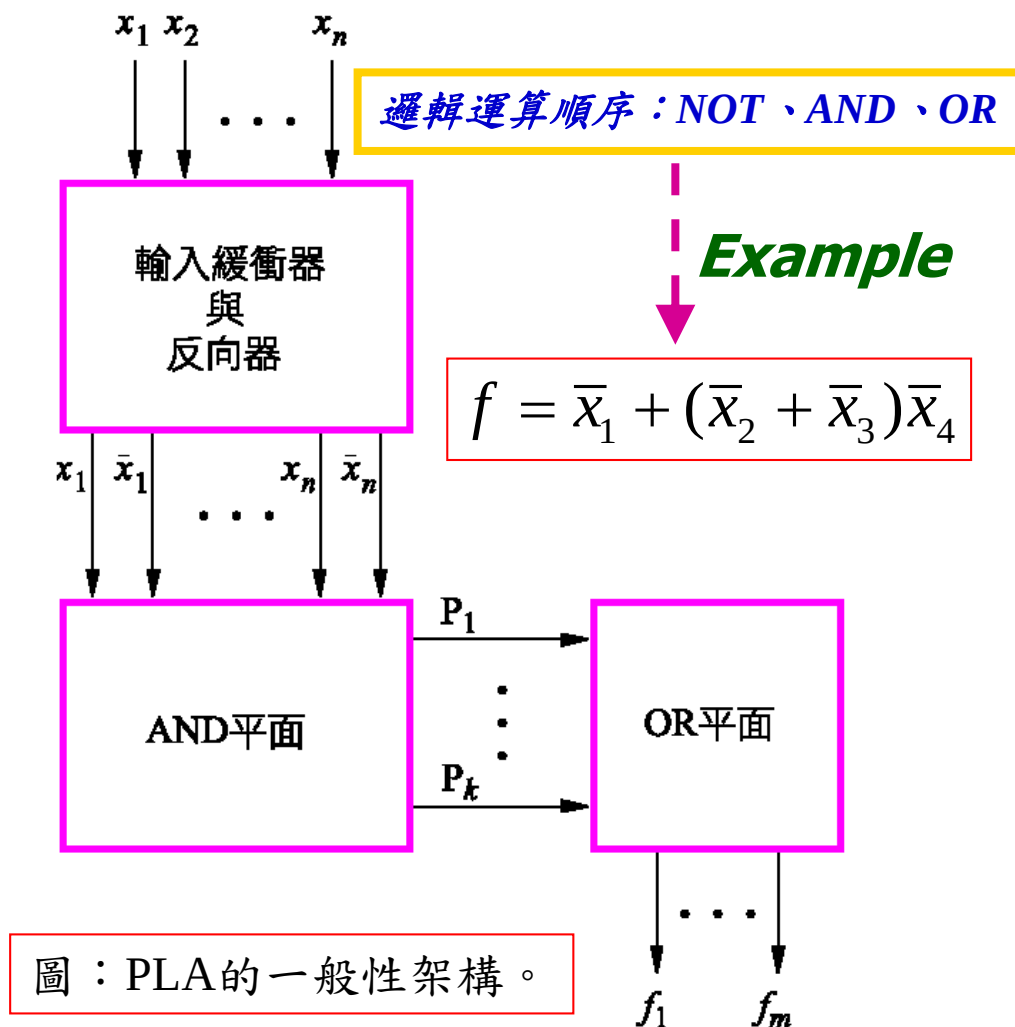
圖：可程式邏輯元件做為黑盒子。



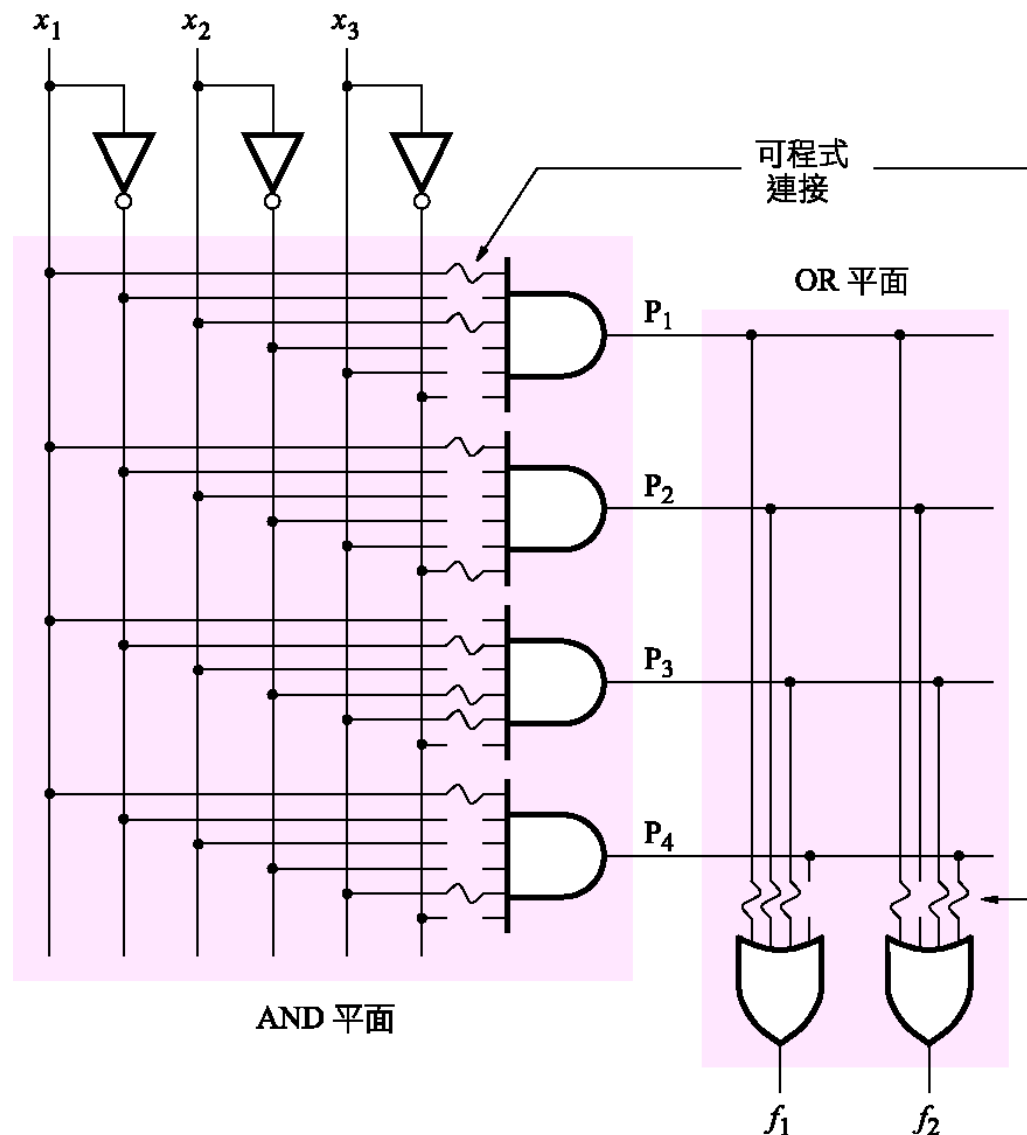
# 可程式邏輯陣列

## Programmable Logic Array (PLA)

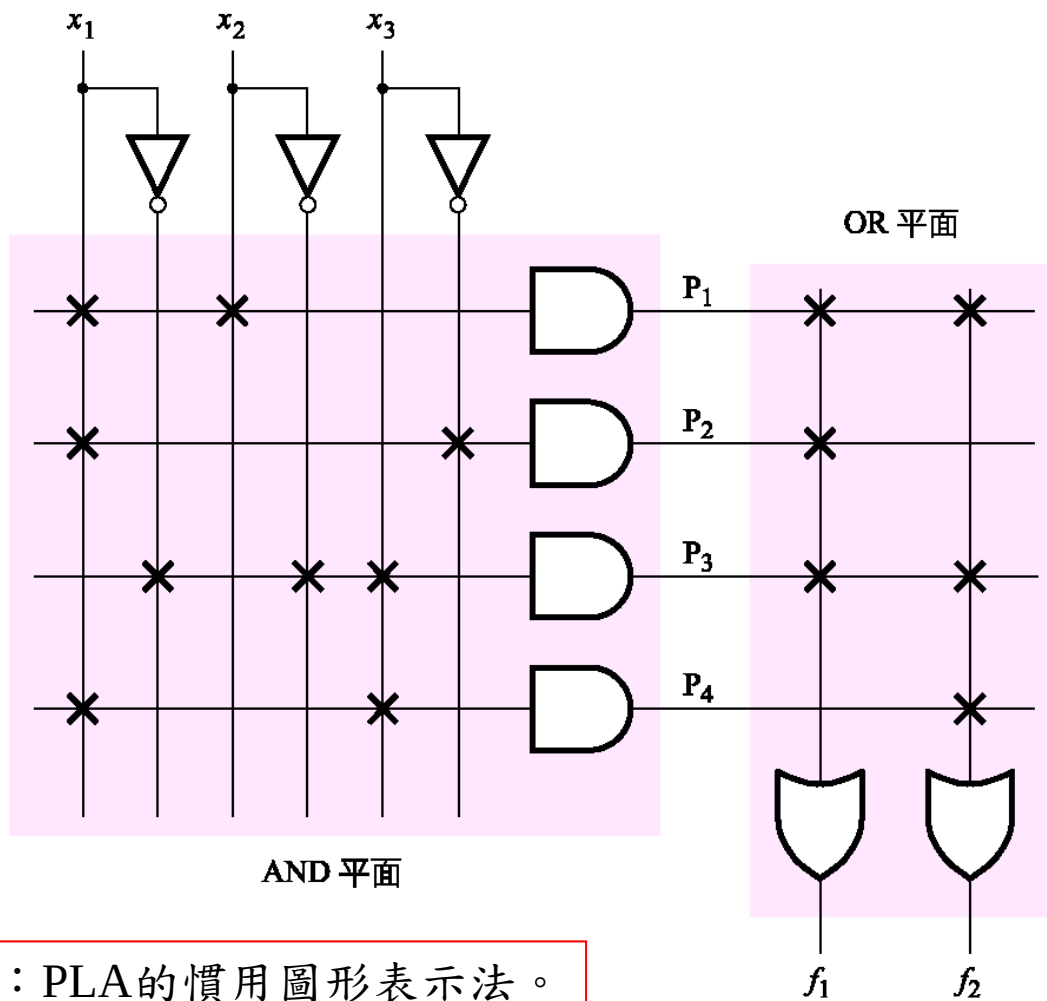
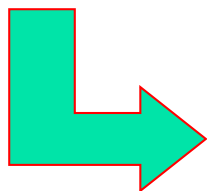
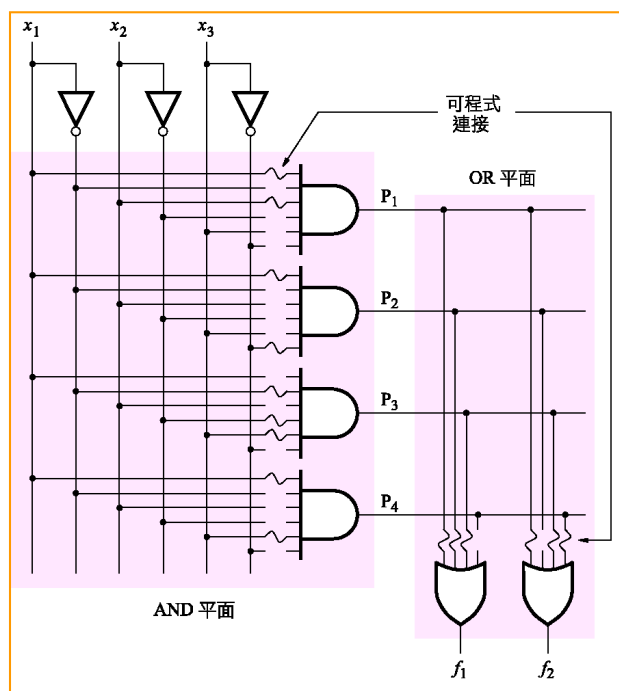
- 首先研發出來的是可程式邏輯陣列 (programmable logic array, PLA)。PLA的一般性架構如右圖所示。
- PLA的輸入經過一組緩衝器（提供每個輸入的真值與互補值）到電路區塊，稱為AND平面 (plane)，或是AND陣列 (array)。
- 其中的每一項都可以設定成實作的任意AND函數。這些乘積項作為OR平面 (plane) 的輸入，產生了輸出。



- 小型PLA更細節的電路圖如右圖所示，其中PLA有三個輸入，四個乘積項，兩個輸出。
- 雖然上一圖清楚描述PLA的函數架構，這種畫法對大型電路來說相當不合適。
- 因此，我們習慣上以右圖的方式來表示。每個AND邏輯閘畫成單一水平線連接到AND邏輯閘符號。
- 實作圖如右圖所示，乘積項所需的可程式連接。



圖：PLA的邏輯閘階層電路圖。

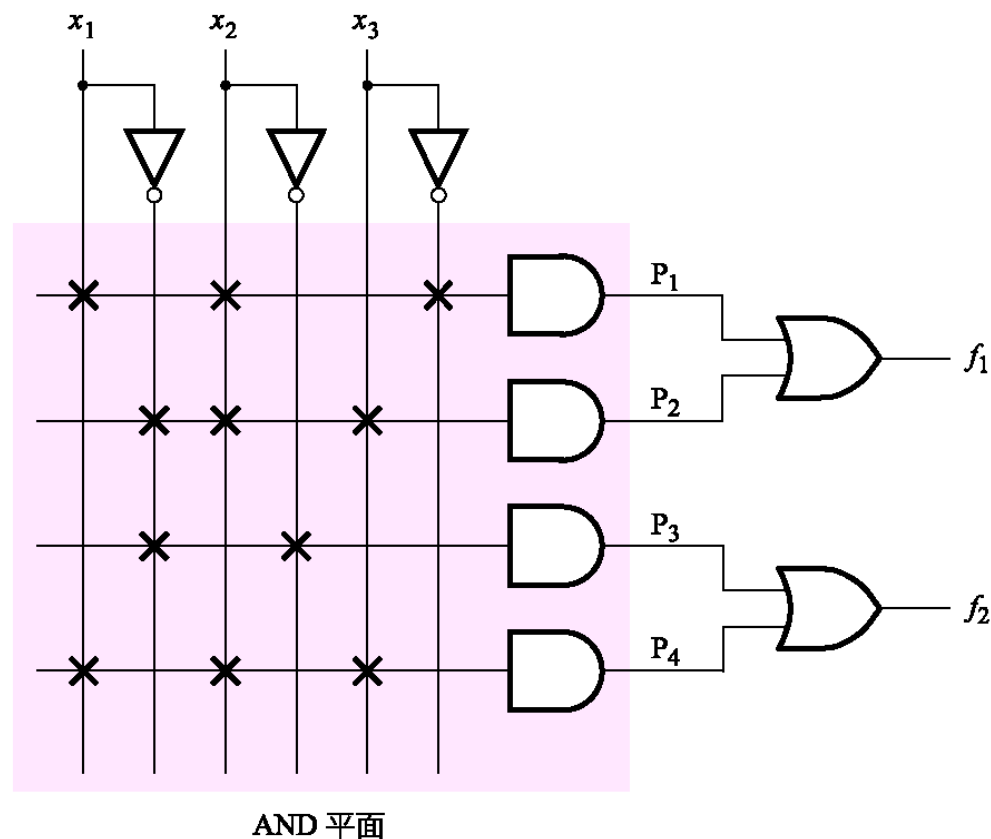


圖：PLA的慣用圖形表示法。

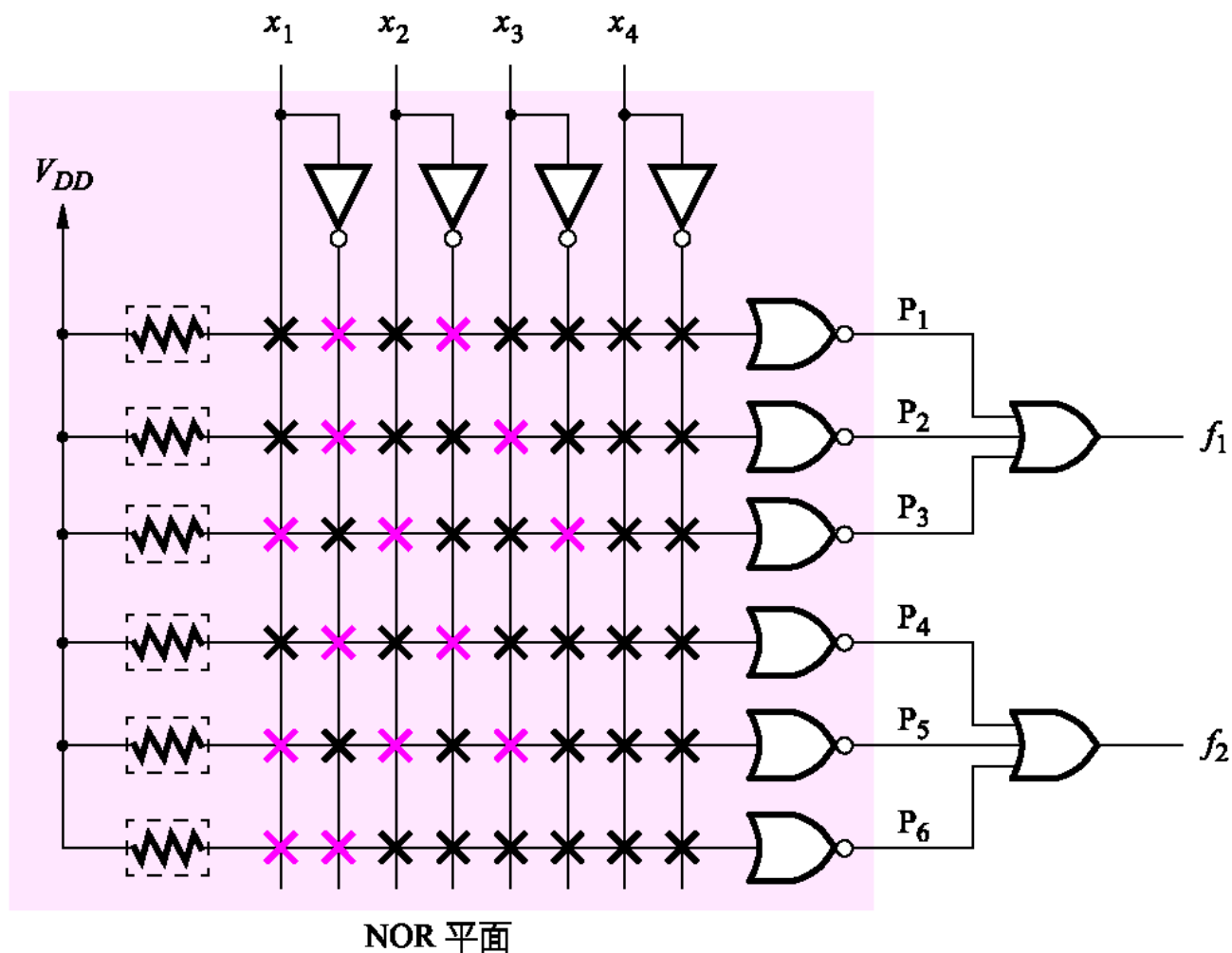
# 可程式陣列邏輯

## Programmable Array Logic (PAL)

- 比較：PLA中AND和OR邏輯閘都是可程式化。
- 可程式開關對元件製造商而言有兩個難題：很難正確地製造，而且降低了實作於PLA上電路的速度效能。
- 這些缺點促使研發相似元件，AND平面為可程式化，但是OR平面為固定的。這種晶片稱為可程式陣列邏輯 (programmable array logic, PAL) 元件。
- 三個輸入，四個乘積項，兩個輸出的PAL例子如右圖所示。



圖：PAL的例子。

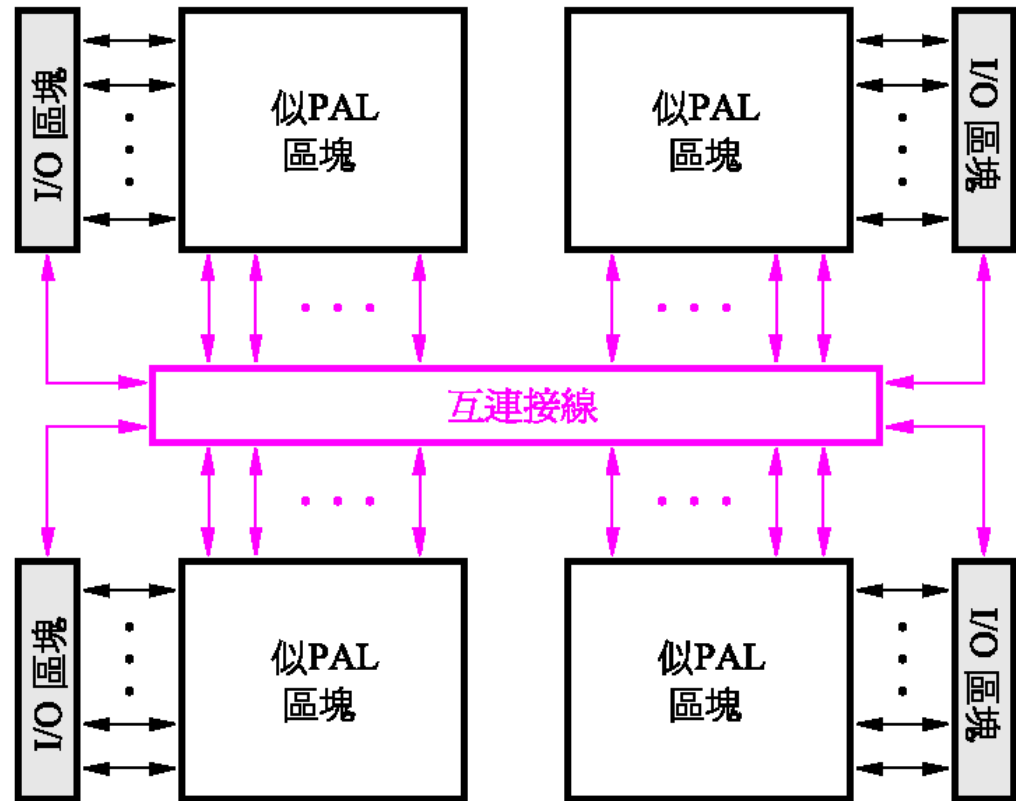


圖：被程式化的PAL實作函數。

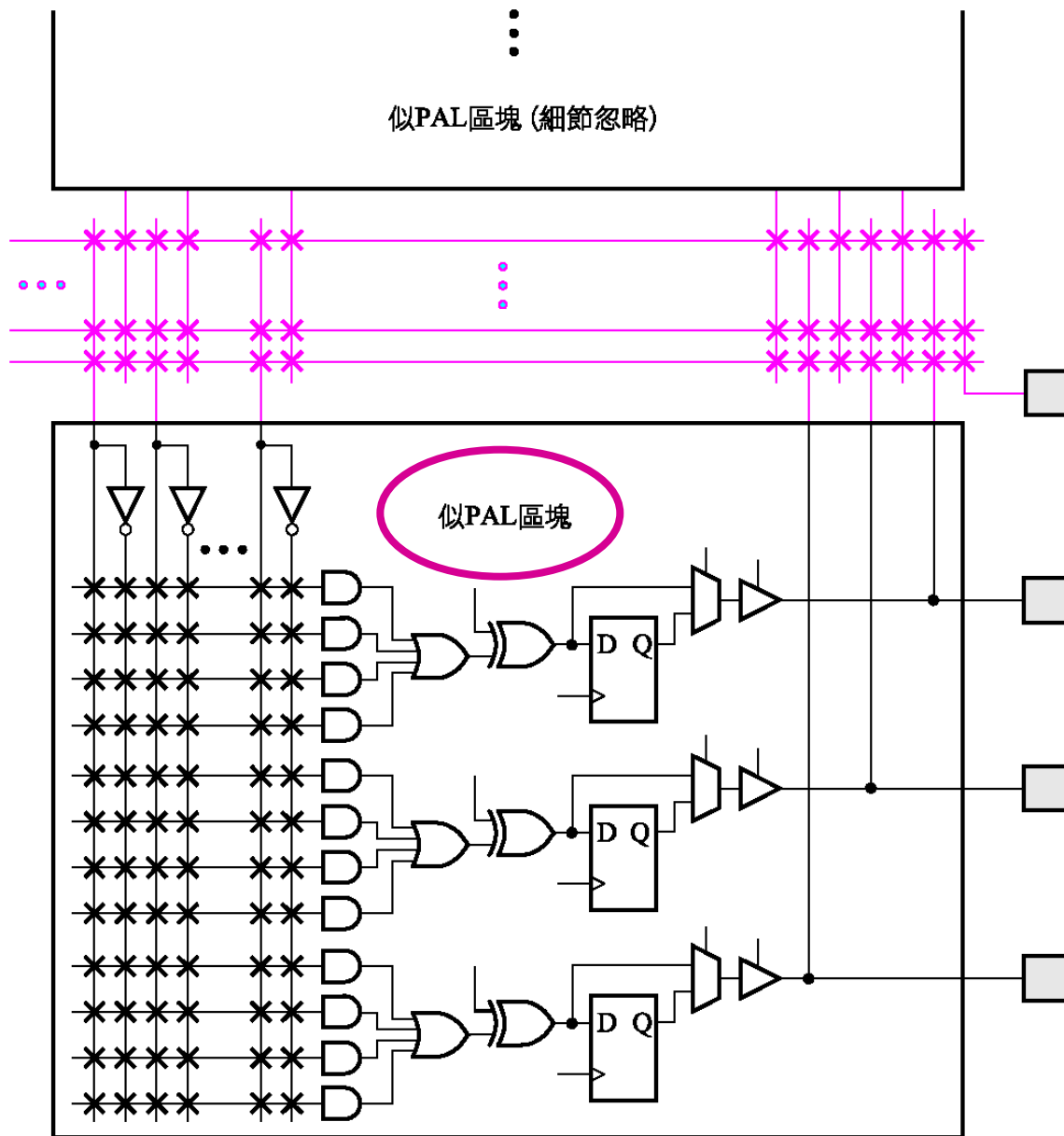
## 複雜可程式邏輯元件

## Complex Programmable Logic Devices (CPLDs)

- 如果要實作更多輸入與輸出的電路，就必須使用多個PLA或PAL，或者是更複雜的晶片，稱為複雜可程式邏輯元件 (complex programmable logic device, CPLD)。
- 每個電路區塊都類似於PLA或PAL；我們稱之為似PAL區塊 (PAL-like block)。
- CPLD的例子如右圖所示，包含四個似PAL區塊，連接到一組互連接線 (interconnection wire)。



圖：複雜可程式邏輯元件 (CPLD) 的架構。



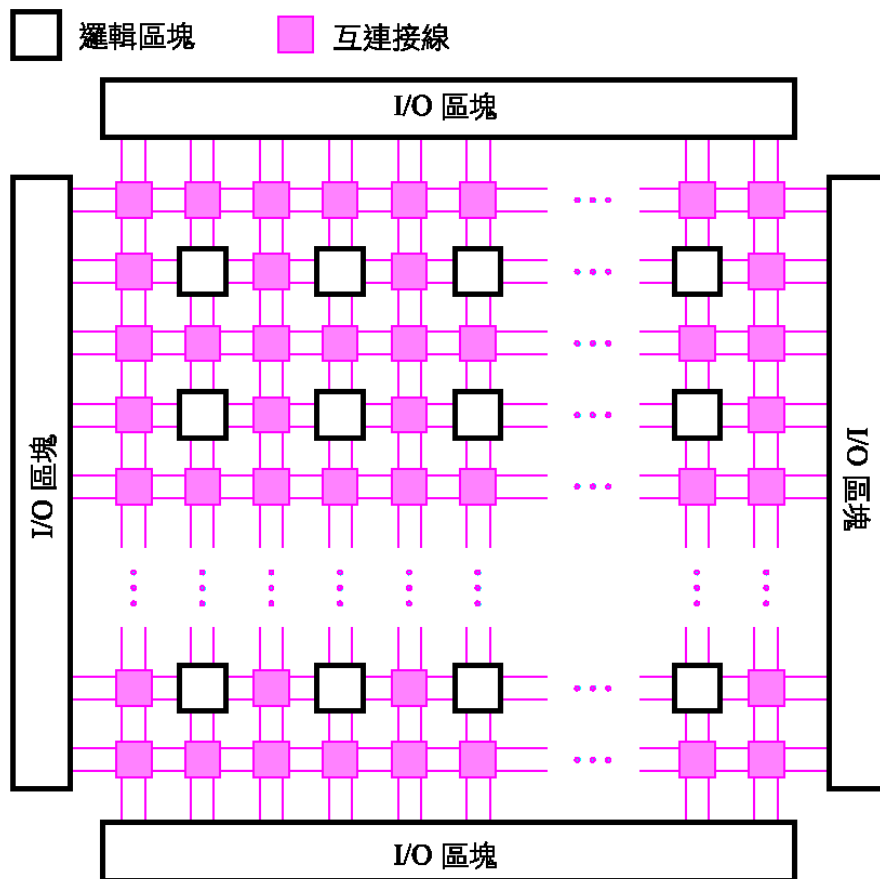
圖：CPLD中的一部份。

# 場域可程式邏輯陣列 2022/09/22

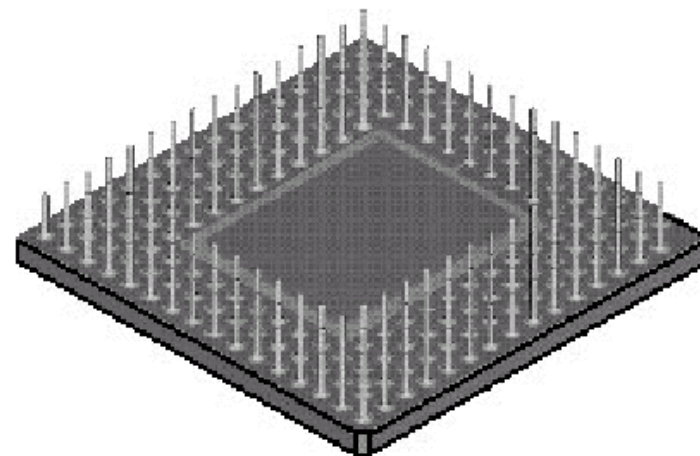
## Field-Programmable Gate Arrays

- 常用的計算方法是建構電路所需雙輸入NAND邏輯閘的總數，此量測方法稱為**等效邏輯閘 (equivalent gate)**。
- **場域可程式邏輯閘陣列 (field-programmable gate array, FPGA)** 是可以實作較大型邏輯電路的可程式邏輯元件。
- **FPGA提供邏輯區塊 (logic block)** 來實作要求的函數。
- **FPGA的一般性架構如圖(a)所示，包含三種資源：邏輯區塊，連到包裝接腳的I/O區塊，以及互連接線與開關。**
- **邏輯區塊是二維陣列，而互連接線是水平與垂直繞線通道 (routing channel)，在邏輯區塊的行與列之間。**
- **圖(a)為兩個可程式開關的位置。**
- **圖(b)為另一種包裝，稱為PGA (pin grid array)。**
- **另一個包裝技術稱為BGA (ball grid array)，和PGA很類似，除了接腳是小圓球，不是柱狀。**
- **最常用的邏輯區塊稱為對照表 (lookup table, LUT)，包含儲存單元 (storage cell) 用來實作小型函數。**





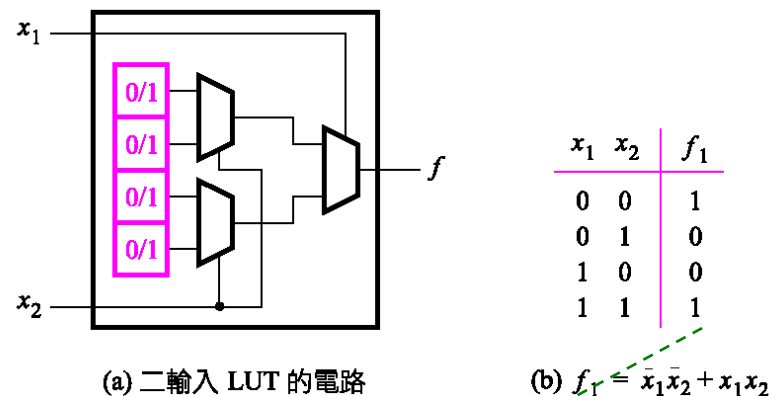
(a) FPGA 的一般性架構



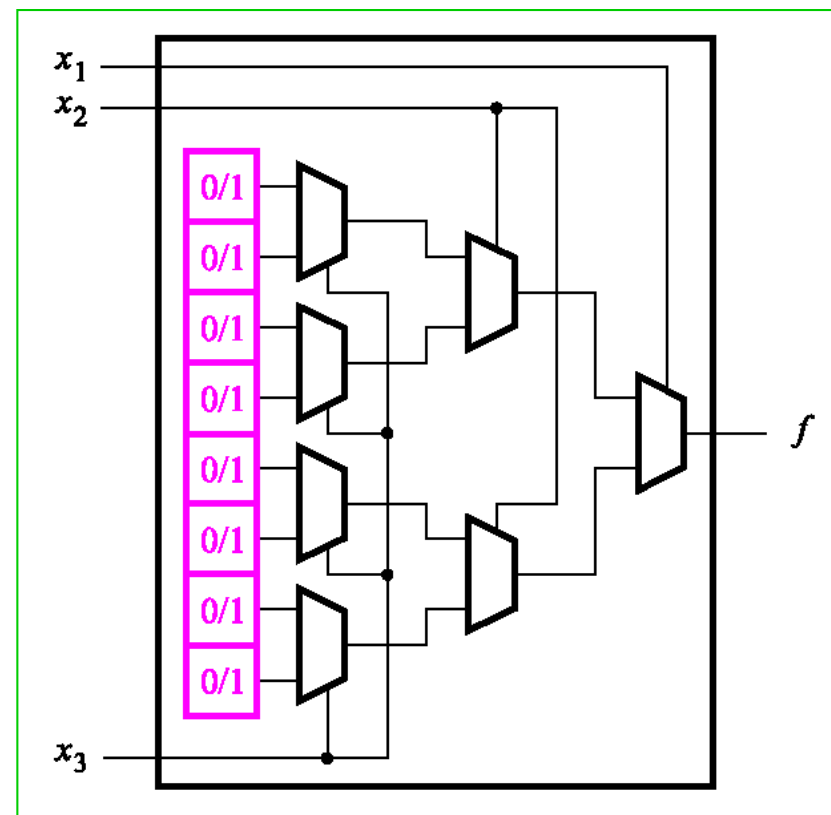
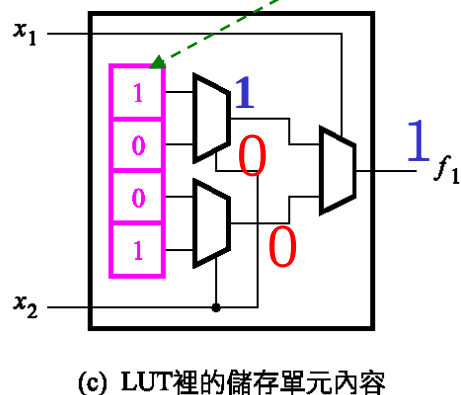
(b) PGA包裝 (底視圖)

圖：場域可程式邏輯閘陣列 (FPGA)。

- 圖(a)為小型LUT的架構，有兩個輸入 $x_1$ 和 $x_2$ ，以及一個輸出 $f$ 。
- 為了瞭解邏輯函數如何實現在二輸入LUT，考慮圖(b)的真值表。
- 根據此表的函數 $f_1$ 可以儲存在LUT中，如圖(c)所示。
- 假設存取儲存單元內容的**唯一方法是透過多工器來實作邏輯函數**。



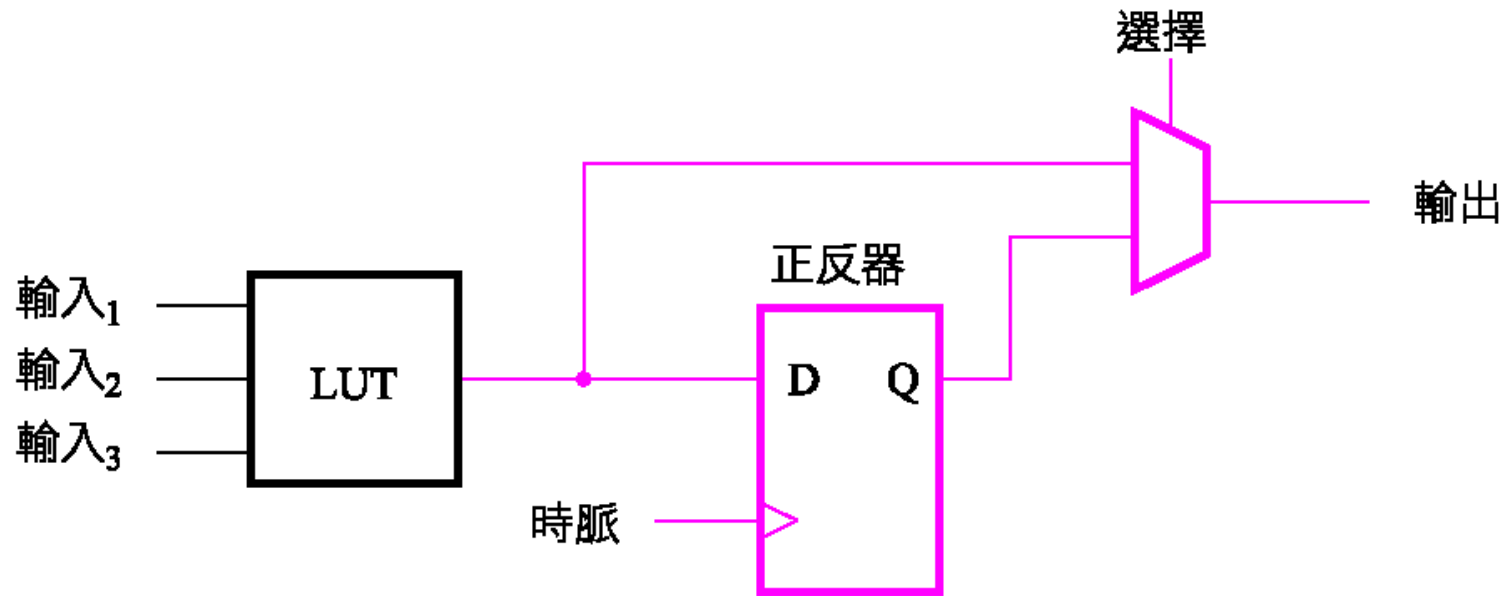
$$(b) f_1 = \bar{x}_1 \bar{x}_2 + x_1 x_2$$



圖：三輸入LUT。

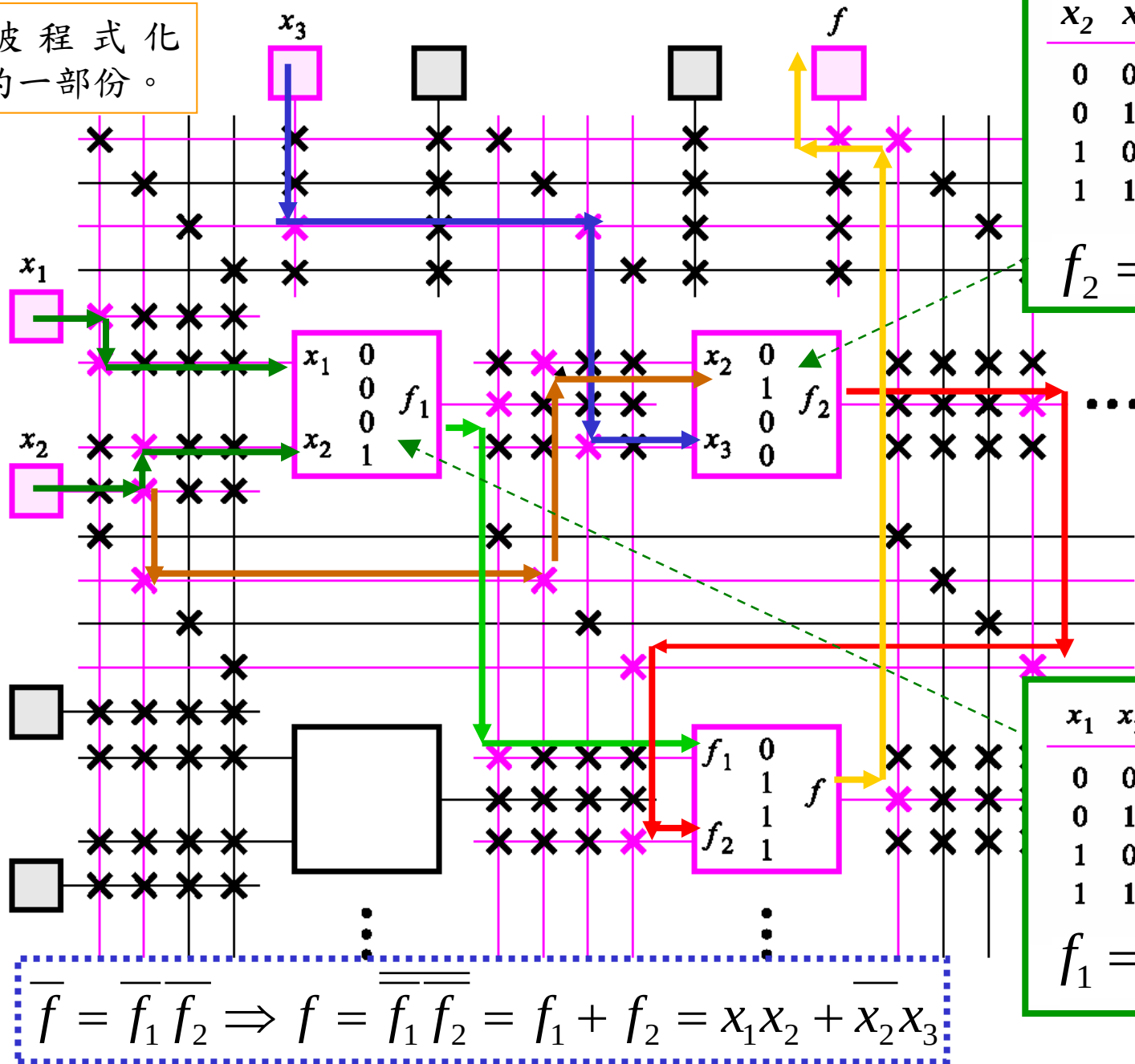
圖：二輸入對照表 (LUT, lookup table)。

- 下圖顯示**正反器**如何包含在**FPGA邏輯區塊**中。
- **FPGA**中**LUT**的儲存單元是**揮發性 (volatile)**，當晶片的電源關掉時儲存的內容將會遺失，因此**每次接上電源時就必須重新對FPGA程式化**。
- 通常有個小型記憶體晶片可以永遠保持其資料，稱為**可程式唯讀記憶體 (programmable read-only memory, PROM)**，包含在放置**FPGA**的電路板上。
- 每當晶片電源接上時，**FPGA**裡的儲存單元會從**PROM**自動載入資料。



圖：FPGA邏輯區塊中包含正反器。

圖：被程式化  
FPGA的一部份。



# 使用XOR邏輯閘

## Use of XOR Gates

2023/09/28

- 兩個變數的XOR函數定義為  $x_1 \oplus x_2 = \bar{x}_1 x_2 + x_1 \bar{x}_2$ 。之前總和位元的式子可以改成只用XOR運算，如下

$$s_i = (\bar{x}_i y_i + x_i \bar{y}_i) \bar{c}_i + (\bar{x}_i \bar{y}_i + x_i y_i) c_i$$

$$= (x_i \oplus y_i) \bar{c}_i + \overline{(x_i \oplus y_i)} c_i$$

$$= (x_i \oplus y_i) \oplus c_i$$

$$s_i = \bar{x}_i y_i \bar{c}_i + x_i \bar{y}_i \bar{c}_i + \bar{x}_i \bar{y}_i c_i + x_i y_i c_i$$

- XOR運算具有結合性，因此可以寫成

$$s_i = x_i \oplus y_i \oplus c_i$$

- 因此可以用一個三輸入XOR邏輯閘來實現 $s_i$ 。

| $c_i$ | $x_i$ | $y_i$ | $c_{i+1}$ | $s_i$ |
|-------|-------|-------|-----------|-------|
| 0     | 0     | 0     | 0         | 0     |
| 0     | 0     | 1     | 0         | 1     |
| 0     | 1     | 0     | 0         | 1     |
| 0     | 1     | 1     | 1         | 0     |
| 1     | 0     | 0     | 0         | 1     |
| 1     | 0     | 1     | 1         | 0     |
| 1     | 1     | 0     | 1         | 0     |
| 1     | 1     | 1     | 1         | 1     |

(a) 真值表

| $c_i \backslash x_i y_i$ | 00 | 01 | 11 | 10 |
|--------------------------|----|----|----|----|
| 0                        |    | 1  |    | 1  |
| 1                        | 1  |    | 1  |    |

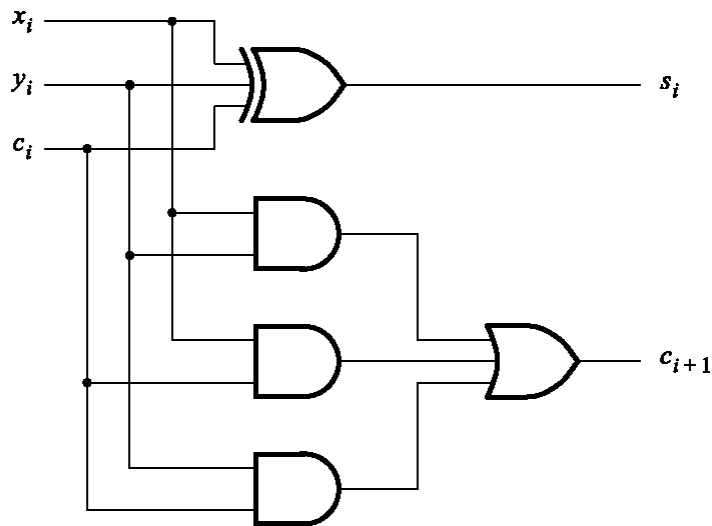
$$s_i = x_i \oplus y_i \oplus c_i$$

| $c_i \backslash x_i y_i$ | 00 | 01 | 11 | 10 |
|--------------------------|----|----|----|----|
| 0                        |    |    | 1  |    |
| 1                        | 1  | 1  | 1  | 1  |

$$c_{i+1} = x_i y_i + x_i c_i + y_i c_i$$

(b) 卡諾圖

$$\begin{aligned} s_i &= (\bar{x}_i y_i + x_i \bar{y}_i) \bar{c}_i + (\bar{x}_i \bar{y}_i + x_i y_i) c_i \\ &= (x_i \oplus y_i) \bar{c}_i + \overline{(x_i \oplus y_i)} c_i \\ &= (x_i \oplus y_i) \oplus c_i \end{aligned}$$

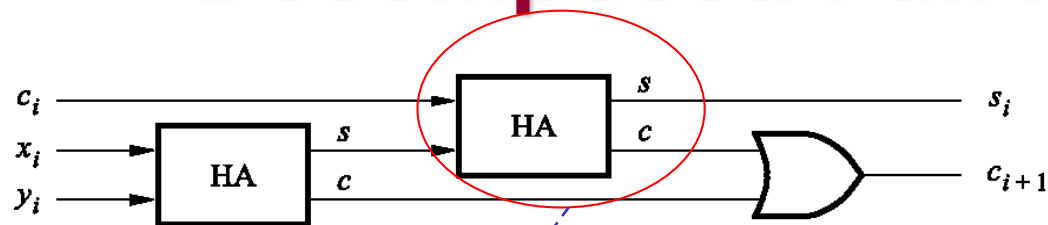


(c) 電路

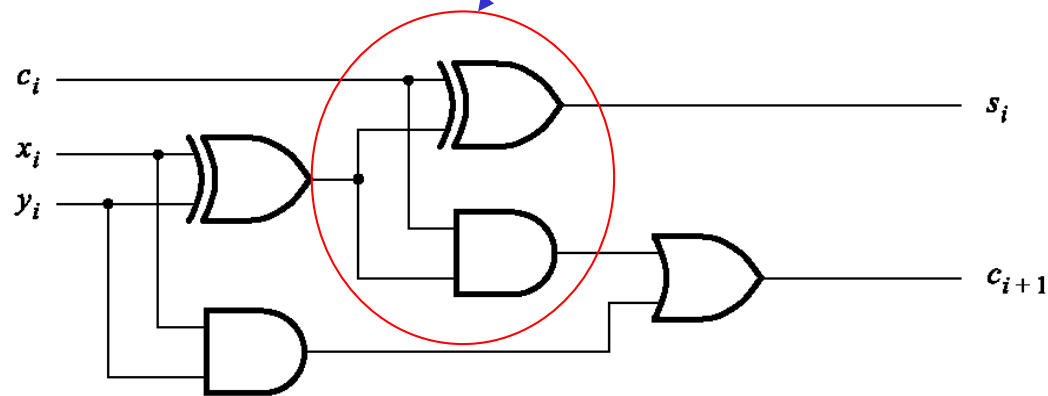
圖：XOR邏輯閘。

# 分解的全加器

## Decomposed Full-Adder



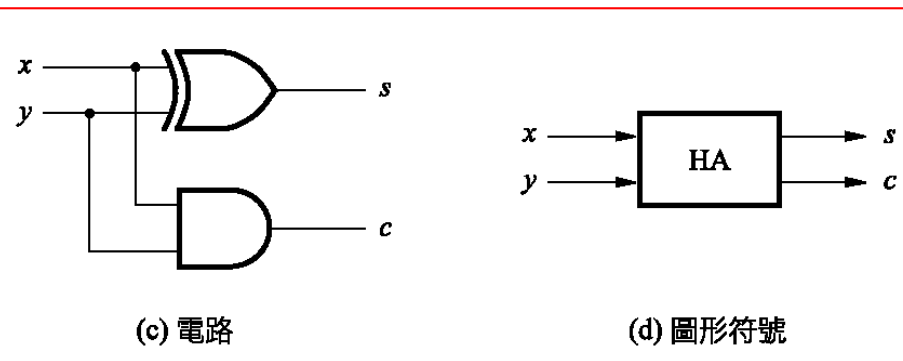
(a) 方塊圖



(b) 詳細圖

| $c_i$ | $x_i$ | $y_i$ | $c_{i+1}$ | $s_i$ |
|-------|-------|-------|-----------|-------|
| 0     | 0     | 0     | 0         | 0     |
| 0     | 0     | 1     | 0         | 1     |
| 0     | 1     | 0     | 0         | 1     |
| 0     | 1     | 1     | 1         | 0     |
| 1     | 0     | 0     | 0         | 1     |
| 1     | 0     | 1     | 1         | 0     |
| 1     | 1     | 0     | 1         | 0     |
| 1     | 1     | 1     | 1         | 1     |

圖：全加器電路的分解實作



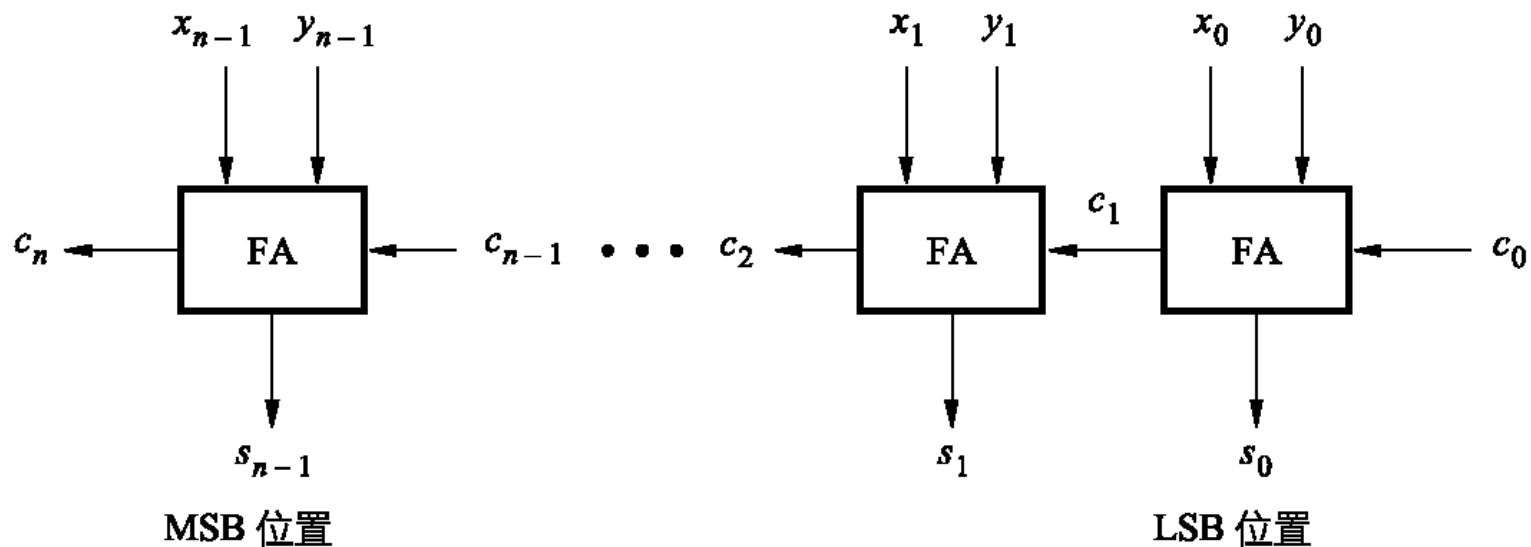
(c) 電路

(d) 圖形符號

# 漣波進位加法器

## Ripple Carry Adder

- 每個位元位置使用全加器電路，連接電路如下圖所示。
- 當運算元 $X$ 和 $Y$ 都套用在加法器的輸入端，需要一點時間才會產生有效的輸出總和 $S$ 。
- 因為進位訊號以類似漣波的方式穿過各級全加器，故該圖的電路稱為漣波進位加法器 (ripple-carry adder)。



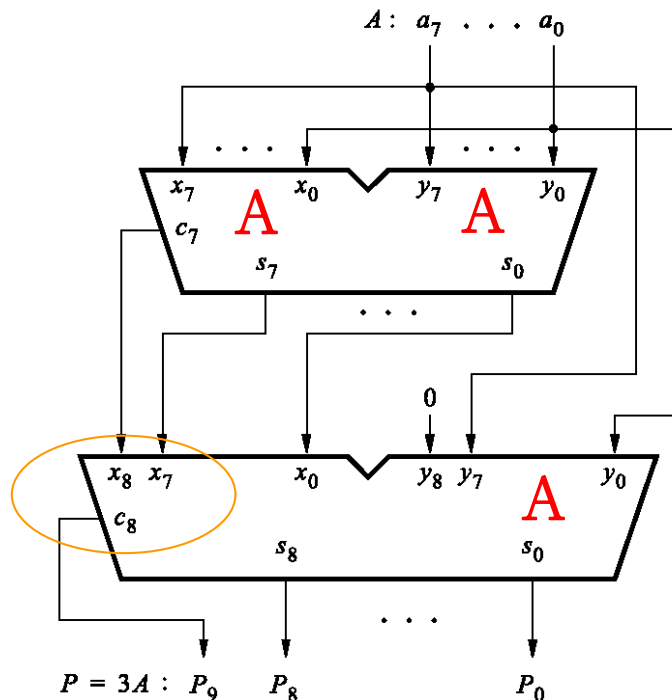
圖： $n$  位元漣波進位加法器。



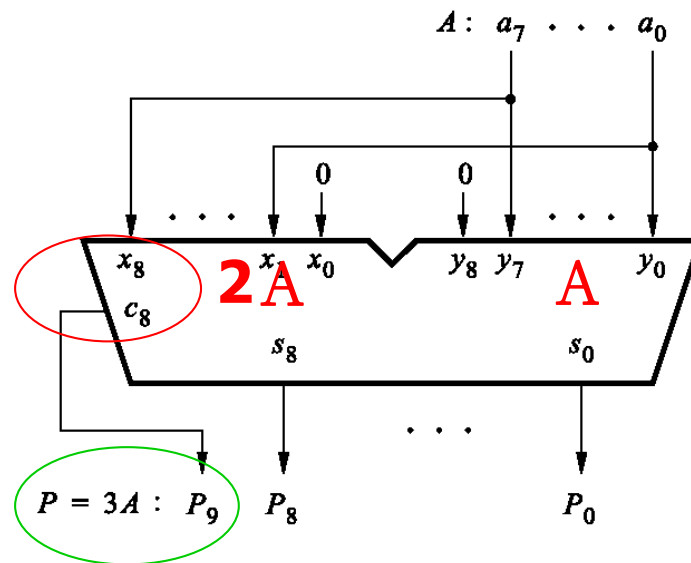
# 設計範例

## Design Example

- 有個簡單的方法可以設計此電路，用兩個漣波進位加法器將三個數字 $A$ 相加，如圖 (a) 所示。
- 第一個加法器產生 $A+A=2A$ （左移），其結果為八個總和位元和來自MSB的進位。第二個加法器產生 $2A+A=3A$ 。
- 只需要一個漣波進位加法器來實作 $3A$ ，如圖(b)所示。



(a) 原始方法



(b) 有效率的設計

# 負數

## Negative Numbers

- 負數可以三種不同方式表示：符號與大小、一的補數以及二的補數。
- 符號可以區別此數為正或負，這方法稱為符號與大小（sign-and-magnitude）數值表示法。
- 同樣的方法也可以用在二進位數值，符號位元對正數和負數分別為0和1。
- 例如，若使用四位元數值，則  $+5 = 0101$  且  $-5 = 1101$ 。
- 因為和十進位符號與大小數值很類似，這種表示法很容易理解。

# 一與二的補數表示法

## 1's and 2's Complement Representation

- 在一的補數 (1's complement) 方法中，欲得到 $n$ 位元負數 $K$ ，可用 $2^n-1$ 減去等效正數 $P$ ；也就是 $K=(2^n-1)-P$ 。
- 在二的補數方法中，欲得到負數 $K$ ，可用 $2^n$ 減去等效正數 $P$ ；也就是 $K=2^n-P$ 。
- 若 $K_1$ 為 $P$ 的一的補數，且 $K_2$ 為 $P$ 的二的補數，則

$$\begin{aligned} K_1 &= (2^n - 1) - P \\ K_2 &= 2^n - P \end{aligned} \Rightarrow \begin{cases} K_1 + P = 2^n - 1 \\ K_2 + P = 2^n \end{cases}$$

- 因此 $K_2=K_1+1$ 。

### **Example:**

**P=0111 (+7)**

**$K_1=(2^4-1)-P=15-7=8=1000$**

**$K_2=2^4-P=16-7=9=1001$**

表：四位元帶正負號整數的說明。

|        | $b_3b_2b_1b_0$ | 符號與大小 | 1的補數 | 2的補數 |
|--------|----------------|-------|------|------|
|        | 0111           | +7    | +7   | +7   |
|        | 0110           | +6    | +6   | +6   |
|        | 0101           | +5    | +5   | +5   |
|        | 0100           | +4    | +4   | +4   |
| 正號不做補數 | 0011           | +3    | +3   | +3   |
|        | 0010           | +2    | +2   | +2   |
|        | 0001           | +1    | +1   | +1   |
|        | 0000           | +0    | +0   | +0   |
| 負號才做補數 | 1000           | -0    | -7   | -8   |
|        | 1001           | -1    | -6   | -7   |
|        | 1010           | -2    | -5   | -6   |
|        | 1011           | -3    | -4   | -5   |
|        | 1100           | -4    | -3   | -4   |
|        | 1101           | -5    | -2   | -3   |
|        | 1110           | -6    | -1   | -2   |
|        | 1111           | -7    | -0   | -1   |

↑

↓

1000

↓

0111(= -7)

↓

1000

↓

-8 (2's)

# 一的補數加法

## 1's Complement Addition

- 如圖左半部所示， $5 + 2 = 7$ 和 $(-5) + 2 = (-3)$ 的計算是很直觀的，運算元的簡單加法就可以得到正確結果。另外兩種情況就不是如此，計算 $5 + (-2) = 3$ 產生位元向量10010。
- 在符號位元的位置有進位輸出，將其加到LSB位置以校正結果，如下圖所示。

$$\begin{array}{r}
 (+5) \quad 0101 \\
 + (+2) \quad +0010 \\
 \hline
 (+7) \quad 0111
 \end{array}$$

$$\begin{array}{r}
 (-5) \quad 1010 \\
 + (+2) \quad +0010 \\
 \hline
 (-3) \quad 1100
 \end{array}$$

$$\begin{array}{r}
 (+5) \quad 0101 \\
 + (-2) \quad +1101 \\
 \hline
 (+3) \quad 10010
 \end{array}$$

$$\begin{array}{r}
 10010 \\
 +1 \\
 \hline
 0011
 \end{array}$$

$$\begin{array}{r}
 (-5) \quad 1010 \\
 + (-2) \quad +1101 \\
 \hline
 (-7) \quad 10111
 \end{array}$$

$$\begin{array}{r}
 10111 \\
 +1 \\
 \hline
 1000
 \end{array}$$

圖：一的補數加法的例子。

# 二的補數加法

## 2's Complement Addition

■ 下圖顯示如何以二的補數執行加法。

$$\begin{array}{r}
 (+5) \quad 0101 \\
 + (+2) \quad + 0010 \\
 \hline
 (+7) \quad 0111
 \end{array}$$

$$\begin{array}{r}
 (-5) \quad 1011 \\
 + (+2) \quad + 0010 \\
 \hline
 (-3) \quad 1101
 \end{array}$$

$$\begin{array}{r}
 (+5) \quad 0101 \\
 + (-2) \quad + 1110 \\
 \hline
 (+3) \quad 10011
 \end{array}$$

↑  
忽略

$$\begin{array}{r}
 (-5) \quad 1011 \\
 + (-2) \quad + 1110 \\
 \hline
 (-7) \quad 11001
 \end{array}$$

↑  
忽略

圖：二的補數加法的例子。

# 二的補數減法

## 2's Complement Subtraction

■ 下圖顯示如何以二的補數執行減法。

$$\begin{array}{r}
 (+5) \quad 0101 \\
 - (+2) \quad -0010 \\
 \hline
 (+3)
 \end{array}
 \Rightarrow
 \begin{array}{r}
 0101 \\
 + 1110 \\
 \hline
 10011
 \end{array}$$

↑  
忽略

$$\begin{array}{r}
 (-5) \quad 1011 \\
 - (+2) \quad -0010 \\
 \hline
 (-7)
 \end{array}
 \Rightarrow
 \begin{array}{r}
 1011 \\
 + 1110 \\
 \hline
 11001
 \end{array}$$

↑  
忽略

$$\begin{array}{r}
 (+5) \quad 0101 \\
 - (-2) \quad -1110 \\
 \hline
 (+7)
 \end{array}
 \Rightarrow
 \begin{array}{r}
 0101 \\
 + 0010 \\
 \hline
 0111
 \end{array}$$

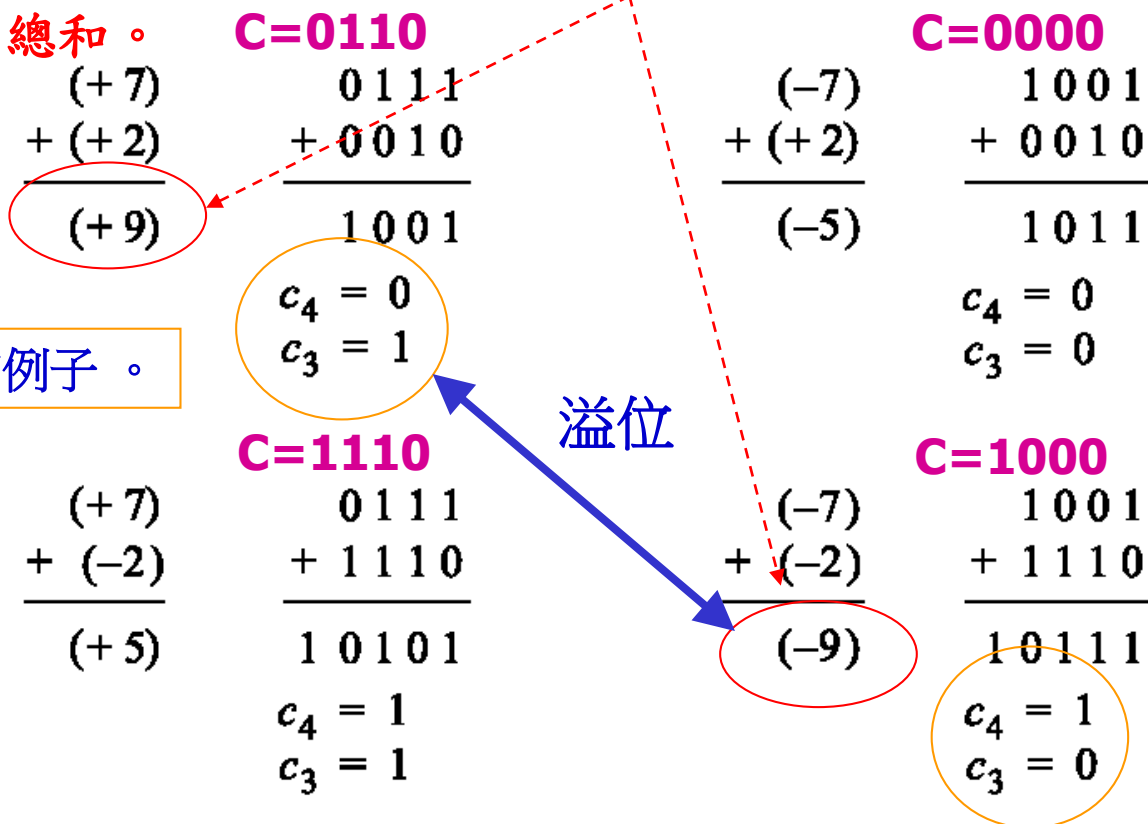
$$\begin{array}{r}
 (-5) \quad 1011 \\
 - (-2) \quad -1110 \\
 \hline
 (-3)
 \end{array}
 \Rightarrow
 \begin{array}{r}
 1011 \\
 + 0010 \\
 \hline
 1101
 \end{array}$$

圖：二的補數減法的例子。

# 算術溢位(1/2)

## Arithmetic Overflow

- 若以 $n$ 位元表示帶正負號數，則結果必須在範圍 $-2^{n-1}$ 到 $2^{n-1}-1$ 。若結果不在這範圍內，則稱為發生**算術溢位** (arithmetic overflow)。
- 大小為7和2的二的補數，相加的四種情況如下圖所示。
- 圖中顯示當進位輸出有不同值時，會發生溢位；當有相同值時，則產生正確的總和。



圖：決定溢位的例子。



## 算術溢位(2/2)

### Arithmetic Overflow

- 我們作個快速的確認，考慮上圖的例子，這些數值夠小使得任何情況都不會發生溢位。在圖上方的兩個例子，符號和MSB位置都有進位輸出0。在圖下方的兩個例子，符號和MSB位置都有進位輸出1。因此，可以下列方式偵測溢位的發生

$$\begin{aligned}\text{溢位} &= c_3 \bar{c}_4 + \bar{c}_3 c_4 \\ &= c_3 \oplus c_4\end{aligned}$$

- 對 $n$  位元數值而言

$$\text{溢位} = c_{n-1} \oplus c_n$$